

User Behavior Simulation with Large Language Model-based Agents

LEI WANG, JINGSEN ZHANG, and HAO YANG, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

ZHI-YUAN CHEN, Renmin University of China, Beijing, China

JIAKAI TANG, ZEYU ZHANG, and XU CHEN, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

YANKAI LIN, Renmin University of China, Beijing, China

HAO SUN and RUIHUA SONG, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

XIN ZHAO, School of Information, Renmin University of China, Beijing, China

JUN XU and ZHICHENG DOU, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

JUN WANG, University College London, United Kingdom of Great Britain and Northern Ireland

JI-RONG WEN, Renmin University of China, Beijing, China

Simulating high quality user behavior data has always been a fundamental yet challenging problem in human-centered applications such as recommendation systems, social networks, among many others. The major difficulty of user behavior simulation originates from the intricate mechanism of human cognitive and decision processes. Recently, substantial evidence has suggested that by learning huge amounts of web knowledge, large language models (LLMs) can achieve human-like intelligence and generalization capabilities. Inspired by such capabilities, in this article, we take an initial step to study the potential of using LLMs for user behavior simulation in the recommendation domain. To make LLMs act like humans, we design profile, memory and action modules to equip them, building LLM-based agents to simulate real users. To enable interactions

Authors' Contact Information: Lei Wang, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: wanglei154@ruc.edu.cn; Jingsen Zhang, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: zhangjingsen@ruc.edu.cn; Hao Yang, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: hao.yang@ruc.edu.cn; Zhi-Yuan Chen, Renmin University of China, Beijing, China; e-mail: zhiyuan.chen2001@gmail.com; Jiakai Tang, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: tangjiakai5704@ruc.edu.cn; Zeyu Zhang, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: zeyuzhang@ruc.edu.cn; Xu Chen (corresponding author), Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: xu.chen@ruc.edu.cn; Yankai Lin (corresponding author), Renmin University of China, Beijing, China; e-mail: mrrlyk423@gmail.com; Hao Sun, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: haosun@ruc.edu.cn; Ruihua Song, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: rsong@ruc.edu.cn; Xin Zhao, School of Information, Renmin University of China, Beijing, China; e-mail: batmanfly@gmail.com; Jun Xu, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: junxu@ruc.edu.cn; Zhicheng Dou, Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China; e-mail: dou@ruc.edu.cn; Jun Wang, University College London, United Kingdom of Great Britain and Northern Ireland; e-mail: jun.wang@cs.ucl.ac.uk; Ji-Rong Wen (corresponding author), Renmin University of China, Beijing, China; e-mail: jrwen@ruc.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1558-2868/2025/1-ART55

<https://doi.org/10.1145/3708985>

between different agents and observe their behavior patterns, we design a sandbox environment, where each agent can interact with the recommendation system, and different agents can converse with their friends *via* one-to-one chatting or one-to-many social broadcasting. In the experiments, we first demonstrate the believability of the agent-generated behaviors based on both subjective and objective evaluations. Then, to show the potential applications of our method, we simulate and study two social phenomena including (1) information cocoons and (2) user conformity behaviors. We find that controlling the personalization degree of recommendation algorithms and improving the heterogeneity of user social relations can be two effective strategies for alleviating the problem of information cocoon, and the conformity behaviors can be highly influenced by the amount of user social relations. To advance this direction, we have released our project at <https://github.com/RUC-GSAI/YuLan-Rec>.

CCS Concepts: • **Information systems** → **Recommender systems**; **Language models**;

Additional Key Words and Phrases: recommender system, large language model, user simulation

ACM Reference format:

Lei Wang, Jingsen Zhang, Hao Yang, Zhi-Yuan Chen, Jiakai Tang, Zeyu Zhang, Xu Chen, Yankai Lin, Hao Sun, Ruihua Song, Xin Zhao, Jun Xu, Zhicheng Dou, Jun Wang, and Ji-Rong Wen. 2025. User Behavior Simulation with Large Language Model-based Agents. *ACM Trans. Inf. Syst.* 43, 2, Article 55 (January 2025), 37 pages. <https://doi.org/10.1145/3708985>

1 Introduction

Human-centered **artificial intelligence (AI)** strives to advance the techniques that serve humanity, garnering substantial attention from both industry and academic communities over the past few decades. User behavior analysis is a fundamental aspect of human-centered AI, particularly in applications such as recommender systems, social networks, and search engines. It aims to understand and infer human preferences and behavioral patterns, which are essential for optimizing user experience and system performance [1]. The accuracy of user behavior analysis heavily relies on the availability of high-quality user data. However, strict privacy regulations (e.g., general data protection regulation and ethical concerns significantly limit the scope of data sharing. Moreover, the limited availability of diverse and comprehensive datasets makes it difficult to develop and evaluate recommendation systems effectively. These challenges necessitate alternative solutions for obtaining user behavior data, especially those that can simulate real-world scenarios without violating privacy norms [2]. Traditional simulation strategies, such as data-driven and model-driven methods, face notable limitations. Data-driven approaches, while adaptive and accurate, heavily depend on access to high-quality real-world data, raising privacy and ethical concerns. Model-driven methods, on the other hand, require complex rule definitions and often fail to capture the dynamic and nuanced nature of real-world user behaviors [3]. These limitations highlight the need for innovative simulation approaches that balance adaptability, scalability, and realism.

Recently, **large language models (LLMs)** have emerged as powerful tools capable of understanding and generating human-like behaviors by leveraging extensive knowledge encoded from diverse web data [4]. This unique capability makes LLMs well-suited for simulating user behavior, particularly in scenarios requiring subjective decision-making, dynamic interaction patterns, and personalized preferences. Unlike traditional methods, LLM-based simulation can operate without relying on sensitive user data, making it a promising direction for advancing research in privacy-preserving and scalable user behavior analysis. However, transferring the capabilities of LLMs to simulate user behaviors in real-world scenarios is not straightforward. First, users have diverse preferences, personalities, and backgrounds, making it challenging for LLMs to effectively simulate different user personas. Additionally, real-world user behaviors are dynamically interconnected,

with previous actions influencing subsequent ones. Since ordinary LLMs excel at one-shot static tasks, we need to design additional modules to enhance their ability to handle dynamic behavior.

Furthermore, there is a large volume of user behaviors in real-world scenarios, many of which are trivial, such as daily routines like “eating breakfast” or “brushing teeth,” which are not relevant for a recommender system. Simulating every user action is both unnecessary and impractical, so careful consideration is needed to determine which behaviors should be prioritized. Beyond these single-user challenges, designing an environment and execution protocols to effectively organize interactions between multiple users requires significant effort.

To solve the above problems, we propose a novel recommender simulator called *RecAgent*. From the user perspective, we simulate each user with an LLM-based autonomous agent, which is composed of a profiling module, a memory module, and an action module. The profiling module can flexibly and efficiently generate different agent profiles. The memory module is designed to make user behaviors more consistent in dynamic environments, where the previous behaviors of the users can be stored in memory to influence their subsequent decisions. In the action module, we incorporate not only the user behaviors within the recommender system, such as item clicking and browsing, but also consider external factors like friend-chatting and social-advertisement to more comprehensively simulate the user decision process. By incorporating all these modules, we aim to achieve more consistent, reasonable, and reliable user behavior simulation. From the system perspective, our simulator consists of a maximum of 1,000 agents. This configuration balances the need for realistic large-scale simulations with practical considerations, such as the computational time and resource demands associated with LLM inference. It is executed in a round-by-round manner. In each round, the agents autonomously perform actions based on their predefined activity levels. To facilitate human-agent collaboration, we allow a real human to play as an agent in the simulator, and interact with the recommender system and the other agents. Additionally, one can also actively intervene the system by pausing the simulation process, modifying agent profiles, and then re-running the simulator. Such intervention can be beneficial for studying the effects of emergency events, user counterfactual behaviors, and so on.

Compared with traditional recommendation simulation strategies, our simulator uses LLMs to capture the user decision processes. Since LLMs have learned comprehensive web knowledge, they could be more effective in simulating the recommender system, which is a typical web application. In addition, our simulator does not need additional data to initialize the simulation process, which makes it possible to jointly simulate multiple scenarios. However, in traditional simulation strategies [3], data-driven methods are more adaptive and accurate, but need real-world data to initialize the simulator, while model-driven methods need to manually define complex rules, which are not scalable and effective.

To evaluate the effectiveness of our simulator, we conduct extensive experiments from both agent and system perspectives. From the agent perspective, we first focus on the evaluation of the memory module, since it is the key to driven agent behaviors. Then, we evaluate the agent as a whole and study whether it can produce believable user behaviors. From the system perspective, we focus on the evaluation of the simulation efficiency and whether actively intervening the simulator can result in expected user behaviors. Finally, we show the applications of our simulator by using it to study two phenomena including (1) information cocoons and (2) user conformity behaviors.

In summary, the main contributions of this article can be concluded as follows:

- We open the direction of user behavior simulation with LLM-based agents in the recommendation domain.
- As an initial attempt in this direction, we design a unified agent framework and a multi-agent environment to simulate real user behaviors.

- We conduct extensive experiments to demonstrate the believability of the simulated user behaviors from our simulator.
- We show the potential of our simulator by studying the phenomena of information cocoons and user conformity behaviors.

2 Related Work

2.1 Recommender Systems

In the age of information explosion, recommendation systems have become an indispensable part of our lives. Recommendation systems can proactively suggest products to users based on their historical behavior and personal preferences, thus addressing the issue of information overload [5–7]. In the past few decades, people have designed a large amount of recommender models, focusing on different settings and scenarios. In specific, content-based algorithms are among the earliest methods employed in recommendation systems. They recommend products similar to those the user has previously shown interest in [8, 9]. **Collaborative filtering (CF)** is the most classic recommendation algorithm, which relies on the similarity between users and items to make recommendations [10, 11]. In specific, user-based CF relies on the historical behavior of users, recommending items liked by other users with similar interests to the target user [12]. Item-based CF, on the other hand, relies on item similarity, recommending items that are similar to those liked by the user [13]. Sequential recommendation focuses on predicting the next behaviors of the users based on their history actions [14, 15]. Graph-based recommendation leverages the graph structure of relationships between items or users to discover potential connections and recommend relevant contents [16, 17]. Conversational recommendation understands user preferences by interactive dialogs, and makes the final recommendation based on all the collected user opinions [18, 19]. Basically, most of the above algorithms aim to design system models to solve the problem of how to generate recommendations. However, we focus on simulating user behaviors, that is, how different users may behave in a recommender system, for example, given a recommendation list, how they make selections.

2.2 User Simulation for Recommender System

User modeling and simulation has always been a crucial aspect of research in recommendation systems [2, 20]. Enabling interactions between simulated users and recommendation systems to generate synthetic data can help in constructing simulators to address issues such as the high cost and risk of training and testing recommendation systems in online environments, as well as dealing with sparse historical data [2, 21, 22]. It also facilitates research into counterfactual questions, such as exploring “what-if” scenarios to investigate the effects of specific interventions on the recommender system and user behavior [23–25]. RecSim [26], which leverages reinforcement learning, focuses on simulating user behavior in real-world situations and can simulate sequential interactive recommendations. RecoGym [27], on the other hand, is a reinforcement learning environment designed for recommender system, modeling traffic patterns of users in e-commerce, and can support both online and offline experiments. In contrast to previous RL-based approaches, Virtual Taobao [28] is trained on real data from Taobao. It generates a virtual user each time to decide whether to click on the item recommended by the recommender system.

Simulation is also a widely utilized method for evaluating the quality and exploring the mechanisms of recommendation and information retrieval algorithms. Sun et al. [29] proposed a method for evaluating dialogue systems by simulating user satisfaction and published the labeled dataset USS. MetaSim [30] emulates users’ analogical thinking to generate more human-like dialogues, enhancing the assessment of **task-oriented dialogue systems (TODS)**. Zhang and Balog [69] developed

a user simulator that generates realistic user responses by modeling individual preferences and interaction flows. Yao et al. [31] decomposed the user model into a choice model for selecting items and a preference model for rating items after interaction. Rahdari et al. [32] employed simulation techniques to assess recommender systems with carousel interfaces, highlighting the significance and potential of simulation-based evaluations in systems characterized by complex user interactions. To investigate the longitudinal dynamics of recommender systems, Zhang et al. [33] developed an agent-based simulation framework, facilitating analyzes that would be difficult to perform in real-world environments. Hazrati and Ricci [34] simulated user choices influenced by different recommender systems, examining attributes such as diversity and quality in the simulated choices.

2.3 LLM for User Simulation

The rise of LLMs presents new opportunities for user simulation. The human-like decision-making capabilities of LLMs enable their use as simulacra of real users [35]. For instance, Ebrat and Rueda [36] introduced *Lusifer*, an LLM-based environment designed to simulate user feedback in online recommender systems, facilitating the training and evaluation of reinforcement learning-based recommenders. Similarly, LLM-InS [37] leveraged LLMs to simulate user patterns and generate interactions, addressing the cold-start problem in item recommendation.

LLMs' advanced language abilities make them particularly suitable for conversational recommendation or search tasks. iEvalLM [38] proposed an interactive evaluation method using LLM-simulated users for **conversational recommender systems (CRS)**. Yoon et al. [39] examined the effectiveness of LLMs as generative user simulators in CRS scenarios, showcasing their potential to produce human-like dialogues that enhance system evaluation. Zhu et al. [40] proposed the CSHI user simulator framework for CRS, which allows human users to influence simulated behaviors or directly interact with the system. Wang et al. [41] analyzed common issues in user simulation for **conversational search (CS)** and proposed QA-enhanced simulation to improve evaluation quality. ConvSim [42] demonstrated the ability to provide feedback and clarifications in multi-turn interactions with mixed-initiative CS systems. Sekulić et al. [43] compared LLM-based user simulations, such as GPT-2 and GPT-3, in CS to align with human query reformulation patterns. AGENT-CQ [44] utilized LLMs to generate clarifying questions by incorporating user characteristics, creating diverse and realistic responses and offering scalable solutions for data generation in CS.

A parallel trend involves using LLMs for user simulation in TODS. Algherairy and Ahmed [45] designed a user simulator by prompting LLMs with initial tasks, user goals, and dialogue history. Davidson et al. [46] employed "gold" examples to guide LLMs in generating robust and diverse user utterances through in-context learning, though this approach is constrained by the limited context window of LLMs. To mitigate this limitation, DuetSim [47] used one LLM to generate responses and another to verify them, iteratively improving output quality. DAUS [48] fine-tuned LLMs on domain-specific datasets with user goals and dialogue history to create domain-aware simulators.

Unlike previous studies, our work simultaneously models user behaviors in both recommender systems and social networks, simulating realistic user interactions across these platforms.

2.4 LLM-empowered Autonomous Agent

The LLM-empowered autonomous agent has recently garnered significant attention [49]. These agents are widely applied in various scenarios and applications, such as autonomous software development [50–52], serving as tools to assist humans in accomplishing specific tasks [53–55], and acting as planners to break down complex tasks and generate executable plans [56–58]. An innovative application involves constructing a simulation environment composed of agents, where

multiple agents can autonomously take action and interact with each other [59, 60]. Researchers are actively exploring the integration of agents with recommendation systems as well. For instance, [61, 62] involve agents at different stages of recommendations to enhance recommendation effectiveness. Distinct from these studies, our focus is on simulating user behaviors and developing comprehensive modules and environments to enable agents to act like real humans.

3 The Designed Simulator

In this section, we first build an agent framework by empowering LLMs with necessary capabilities for user behavior simulation. Then, we detail the system environment, including the simulation protocol, recommendation algorithm, real-human playing and system intervention. We name our simulator as RecAgent.

3.1 The Agent Framework

While LLMs have displayed unprecedented potential in attaining human-like intelligence, many challenges still exist on leveraging them to simulate real user behaviors. To begin with, real users may have diverse preferences, personalities and backgrounds, how to make LLMs simulate different user characters in both effective and efficient manners needs our careful designs. Then, real user behaviors are usually dynamically correlated. However, ordinary LLMs tend to rely on the static knowledge encoded in their parameters, limiting their ability to adapt to the continuously changing contexts and interactions found in dynamic environments. This necessitates the design of additional modules that enhance the LLMs' ability to respond appropriately as the environment changes. In addition, since it is impossible to simulate all user behaviors, determining the key ones to simulate is also an important but not solved challenge. To overcome these challenges, we design an agent framework, including a profiling module, a memory module, and an action module to enhance ordinary LLMs. In the following, we detail these modules.

3.1.1 Profiling Module. User profiles are a central concept in the recommendation domain, as they determine user preferences, personalities, and behavior patterns. In our simulator, each user profile is composed of attributes such as ID, name, gender, age, traits, career, interests, and behavioral features. The *traits* describe the user's personality, such as being "compassionate," "ambitious," or "optimistic," which influence how users interact with recommendations. The *interests* capture the user's preferences for specific item categories, such as "sci-fi movies" or "comedy videos." These attributes serve as the foundation for simulating personalized behaviors in our system.

To better represent user diversity in the recommendation domain, we identify five key behavioral features commonly observed among online users and incorporate them into the user profiles:

- *Watcher*: Users with this feature actively provide feedback and ratings for the items they interact with, playing a crucial role in improving recommendation quality.
- *Explorer*: These users actively seek out items they have heard about, generating detailed interaction logs that reflect exploratory behaviors, often leading to serendipitous discoveries.
- *Critic*: Users with this feature hold high standards and are more likely to critique both the recommendation system and the recommended items, providing valuable insights into system weaknesses.
- *Chatter*: These users frequently engage in private conversations and are highly influenced by friends' recommendations, simulating the impact of social interactions on decision-making.
- *Poster*: Users who enjoy sharing content publicly on social media, contributing to online communities with reviews and insights, modeling the influence of content-sharing behaviors.

ID	Name	Gender	Age	Traits	Career	Interest	Feature
0	David Smith	male	25	compassionate, caring, ambitious, optimistic	photographer	sci-fi movies, comedy movies	Watcher;Critic;Poster
1	David Miller	female	39	Funloving, creative, practical, energetic, patient	writer	action movies, scifi movies, classic movies	Watcher;Explorer;Poster
2	James Brown	male	70	independent, creative, patient, empathetic	engineer	comedy movies, familyfriendly movies, documentaries, thriller movies	Watcher;Critic;Poster
3	Sarah Miller	female	33	independent, compassionate	farmer	romantic movies, comedy movies, classic movies, family-friendly movies	Watcher;Critic;Poster
4	John Taylor	male	68	optimistic	doctor	action movies, thriller movies	Watcher;Poster
5	Sarah Williams	female	51	meticulous	musician	action movies, documentaries, scifi movies, familyfriendly movies	Watcher;Explorer;Chatter
6	James Jones	male	59	practical, funloving, creative, ambitious, caring	farmer	documentaries	Watcher;Poster
7	Jane Brown	female	30	patient, adventurous, funloving, optimistic	doctor	documentaries	Watcher;Explorer;Poster
8	David Jones	male	23	analytical, energetic, introspective, independent	scientist	familyfriendly movies, thriller movies, action movies, sci-fi movies	Poster
9	James Brown	female	20	ambitious, analytical, optimistic, energetic, meticulous	designer	familyfriendly movies, romantic movies	Critic; Chatter
10	James Garcia	male	20	practical, energetic, introspective, patient	engineer	documentaries, thriller movies, comedy movies, classic movies, romantic movie	Watcher; Explorer; Poster

Fig. 1. Examples of the complete user profiles.

We provide several examples of complete user profiles in Figure 1, illustrating how these attributes combine to create rich and realistic user personas. Additionally, user profiles can be partially specified or left empty, depending on specific simulation requirements, allowing for flexibility in system design.

To generate user profiles efficiently, we design the following three strategies, catering to different simulation needs:

- Handcrafting Method*: This approach manually defines each agent’s profile. For instance, an agent could be described as “David Smith, a 25-year-old male photographer.” This method offers complete flexibility, enabling users to simulate specific types of personas or test edge cases.
- GPT-based Method*: By leveraging LLMs like GPT, this method automates profile creation through carefully crafted prompts. For example, prompts such as “Complete the user’s profile based on the following details” can efficiently generate realistic and diverse user profiles, reducing the time required for large-scale simulations.
- Real-data Alignment Method*: This approach aligns agent profiles with real-world datasets, such as mapping user preferences or demographics based on observed data distributions. This method ensures that simulated user populations reflect real-world trends, enhancing the realism of the simulation process.

These methods are not isolated and can be combined to take advantage of their respective strengths. For example, one might manually specify key attributes of certain profiles while using GPT to fill in less significant details, or align GPT-generated profiles with real-world data to balance diversity and realism.

By crafting detailed and diverse user profiles, the profile module provides the foundation for simulating realistic and dynamic user behaviors. These profiles play a critical role in driving the memory and action modules, ensuring consistency and believability in the simulated interactions.

3.1.2 Memory Module. Recent advancements in cognitive neuroscience [63] highlight three key components of human memory: sensory memory, short-term memory, and long-term memory. Sensory memory captures information directly from the environment and retains it for only a

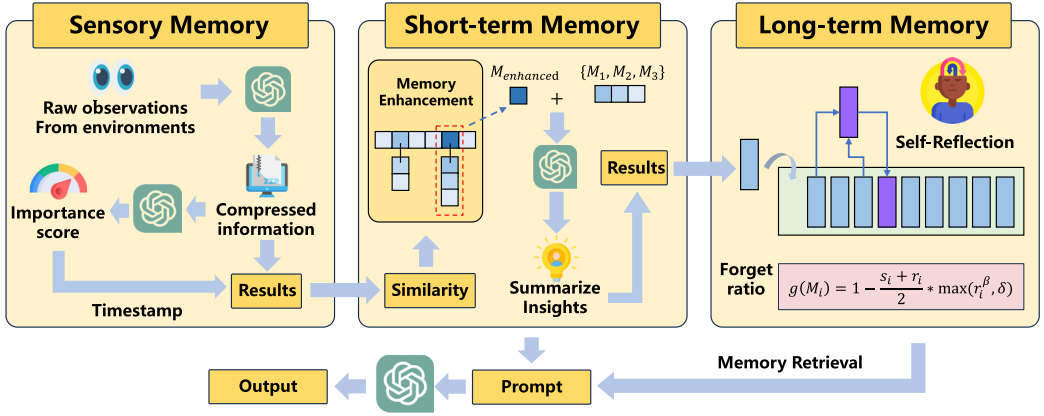


Fig. 2. Overview of the memory module, consisting of sensory, short-term, and long-term memory.

few hundred milliseconds. During this time, significant information is transferred to short-term memory, while less important information is discarded. Short-term memory acts as a bridge, where information can be strengthened through repeated exposure and, eventually, transferred to long-term memory. Long-term memory stores information for extended periods, allowing humans to make decisions based on experiences and produce high-level insights. To accurately model user behaviors, we designed the memory module of our simulator based on the human memory mechanism outlined above, with detailed workings presented in Figure 2. In the following, we first detail the agent sensory memory, short-term memory and long-term memory in our simulator, and then introduce how they work together to accomplish different memory operations.

Sensory Memory. In the simulation process, we define an observation as an event experienced by the agent. For example, an observation could be “David Miller enters recommendation system” or “David Smith watches a movie called Fast & Furious.” The observation is described and recorded directly using natural language, which can retain rich semantic information and is understandable to humans.

All the observations are firstly input into the sensory memory, which are then processed by two sequential steps. In the first step, the observations are compressed into more informative and concise sentences based on LLMs. This step can remove noisy and unimportant contents, which may mislead the agent behaviors. In addition, the compressed memory can save much space to improve the operation efficiency. We present an example of this step in the Appendix A1.

In the second step, we assign each compressed observation with a score ranging from 1 to 10 to indicate its importance. In specific, we consider item-related information to be more important. For example, “feelings, chatting and posting messages on items” are more important observations than the action of “entering recommendation system.” The importance score is leveraged in the memory reading process, and more important memories are more likely to be recalled. In addition, each observation is also accompanied by its timestamp. Finally, the output from the sensory memory is a triplet $M = \langle c, s, t \rangle$, where c is the content of the compressed observation, s and t are the corresponding importance score and timestamp, respectively.

Short-term Memory. Short-term memory serves as an intermediate module connecting sensory memory to long-term memory. This module determines which information should be transformed into long-term memory, and how to transform this information. Similar to the human memory mechanism, in our simulator, if an agent repeatedly encounters similar observations, then the related short-term memories are enhanced and transformed into long-term memories. Formally,

suppose existing short-term memory is $\text{MEM} = \{M_1, M_2, \dots, M_n\}$, where $M_i = \langle c_i, s_i, t_i \rangle$ is a record output from the sensory memory. For a new record $M_{n+1} = \langle c_{n+1}, s_{n+1}, t_{n+1} \rangle$, we first compute the similarity between c_{n+1} and each c_i in MEM based on their embeddings. If the similarity is higher than a predefined threshold, we add M_{n+1} into M , and label that M_i has been enhanced by M_{n+1} . If all the similarities are lower than the threshold, M_{n+1} is added into M without enhancing any record. After a memory record has been enhanced K times, it is then transformed into long-term memories. More specifically, we first summarize the record along with related ones and then prompt the LLM to refine and enhance the summary into high-level insights. Both the original record and the generated insights are subsequently stored in the long-term memory. The record itself is specific, while the insights are more general. By incorporating both of them, we hope that the agent can acquire specific knowledge without compromising its ability to generalize. We present an example on the insight generation process in the Appendix A1.

Long-term Memory. Long-term memory stores information essential for consistent and reasonable agent behavior across similar or unseen environments. Compared to the other memory types, long-term memory has a larger capacity and a longer retention period. However, to maintain efficiency and prevent memory overload, stored information is subject to a forgetting mechanism. Intuitively, recent and important memories have a lower probability of being forgotten. In specific, according to cognitive neuroscience [64, 65], the forgetting rate of human memory is fast in the beginning, and then gradually slows down over time. Therefore, in our simulator, for a memory M_i , it is forgotten with a probability $g(M_i)$ that is proportional to the following power function:

$$g(M_i) = 1 - \frac{s_i + r_i}{2} * \max(r_i^\beta, \delta),$$

where s_i and r_i are normalized recency and importance scores in $(0.0, 1.0)$, larger values indicate that the memory is more recent and important. By $\max(r_i^\beta, \delta)$, we would like to make important memories produced long time ago can still be recalled. $\delta \in (0.0, 1.0)$ is a strength parameter determining when to consider the power function of r_i^β . Extremely, if $\delta \rightarrow 1.0$, the power function is inactivated, and $g(M_i)$ is only determined by the average between the recency and importance scores. $\beta > 1.0$ is a hyper-parameter controlling the power function shape. For more intuitively understanding this function, we present its curves with different hyper-parameters in Figure 15 of Appendix A.3.

Memory Operation. Based on the collaboration between the above three types of memories, the agents can accomplish the following memory operations to support their behaviors:

- *Memory Writing:* Observations are first processed in sensory memory to remove noise and assign importance scores. Processed observations are stored in short-term memory, where repeated exposure leads to enhancement. Once enhanced K times, the observations are transferred to long-term memory along with generated insights.
- *Memory Reading:* To retrieve information relevant to the current observation, the agent queries long-term memory for the top-N most similar records while also incorporating all short-term memory records. This combination allows the agent to simultaneously consider general preferences and recent activities, aligning with common practices in recommendation systems [14, 15].
- *Memory Reflection:* The agent generates high-level insights based on long-term memory records. Following [59], LLMs produce salient questions from recent records, retrieve related memories, and summarize them into insights. Unlike prior methods, our approach merges similar insights and memories to conserve memory space and eliminate redundancy.

By integrating these memory operations, the agents can simulate dynamic decision-making processes and produce realistic behaviors in response to their environments. The collaborative design of sensory, short-term, and long-term memories ensures consistency, adaptability, and efficiency in the simulation.

3.1.3 Action Module. In our simulator, we focus on three types of user behaviors: recommendation-related behaviors, one-to-one chatting behaviors, and one-to-many broadcasting behaviors. For *recommendation-related behaviors*, we consider the following specific agent actions:

- *Searching Behaviors*: Agents actively search for items of interest, reflecting their proactive engagement with the system.
- *Browsing Behaviors*: Agents passively receive recommendations from the system, simulating scenarios where users explore suggestions without specific goals.
- *Clicking Behaviors*: Agents select items they wish to watch or purchase, demonstrating clear interest in particular recommendations.
- *Next-page Behaviors*: When agents are unsatisfied with the current recommended or searched items, they trigger this behavior to request more results.

For *one-to-one chatting behaviors*, we aim to simulate information exchanges between two users. These behaviors can represent online conversations (e.g., via platforms like Twitter or WeChat) or offline discussions in a setting like a coffee shop. In this context, users are allowed to discuss topics specifically related to movies. For simplicity, messages are automatically read by the receiving user upon delivery. For *one-to-many broadcasting behaviors*, we simulate the dissemination of information from one user to multiple others. This includes activities such as posting opinions on social media or sending business advertisements. To streamline the process, users are assumed to read received messages by default.

To effectively guide the above agent actions, we design a unified prompting framework, which comprises the following four components:

- *Profile Information*: This delineates the agent’s role by including attributes such as age, gender, personality traits, and more. For different behaviors, we use ChatGPT to filter the complete profile information, retaining only the most relevant parts.
- *Memory Information*: This includes knowledge extracted from both short-term and long-term memories, contextualized to the current behavior.
- *Instruction*: This provides clear behavioral guidelines. For instance, if the behavior involves expressing feelings about a movie, the instruction might be: “*Imagine you are [agent-name]. How would you feel about this movie you just watched? Please share your personal feelings about the movie in one sentence.*” To enhance the agent’s ability to follow instructions, several few-shot examples can be incorporated into the prompt.
- *Context*: This describes the current state of the agent and the environment, such as the timing of the behavior or relevant external conditions.

We provide detailed examples of the full prompts in the Appendix. This unified prompting framework ensures that agent behaviors are consistent, context-aware, and reflective of the scenarios being simulated.

3.2 The System Environment

In the above section, we focus on introducing a single agent, illustrating its major components. In this section, we take a global perspective, and discuss four crucial aspects of the multi-agent

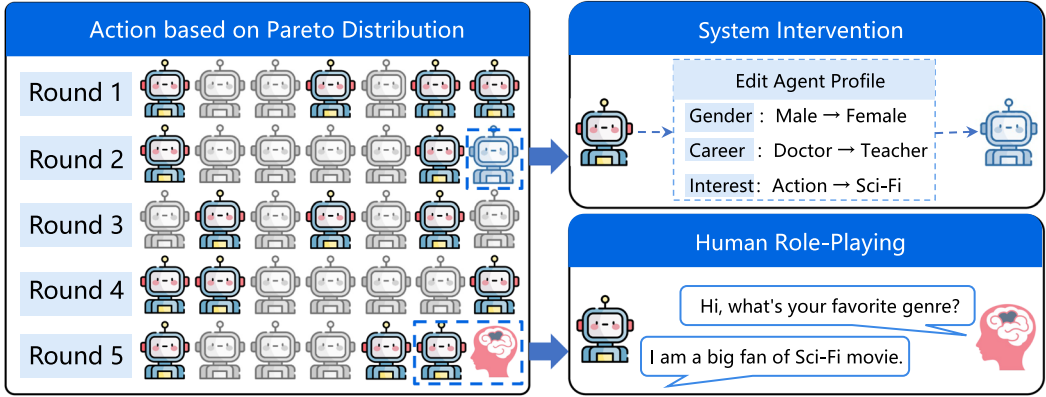


Fig. 3. Key characters of the simulator. Different agents behave in a round-by-round manner based on Pareto distribution, where, in each round, only a small number of agents (labeled in colorful) take actions. One can actively change the simulation process by, for example, altering the agent properties, and can also participate into the simulation process to interact with the recommender system or other agents.

environment that the agents situated within, including the execution protocol, recommendation algorithm, real-human playing, and system intervention (see Figure 3 for an overview).

3.2.1 Round-based Execution. Our simulator operates in a round-by-round manner. During each round, the agents autonomously decide whether they would like to take actions. In real-world scenarios, different users may take actions (e.g., accessing the recommender system or social media) with various activity levels. From a global perspective, these activity levels usually follow a long-tail distribution,¹ meaning that a small number of individuals are highly active, while the majority of users have low action frequency. In order to incorporate such a characteristic into our simulator, we model the activity levels of different agents according to a Pareto distribution.² In specific, the probability density of the activity level is represented as follows:

$$p(x) = \frac{\alpha x_{min}^\alpha}{x^{\alpha+1}},$$

where x_{min} is the minimum activity level, and α is used to control the shape of the distribution.

To assess whether $p(x)$ can be powerful enough to characterize real-world scenarios, we employ it to fit four well-known recommendation datasets including MovieLens, Amazon-Beauty, Book-Crossing, and Steam,³ where we approximate the interaction numbers of the users as their activity levels. The results are illustrated in Figure 4. We can see, by varying the values of α , $p(x)$ can effectively model the long-tail activity distribution of the real-world datasets.

3.2.2 Recommendation Algorithm. In our simulator, the recommendation algorithms are completely separated from the simulator, which facilitates flexible exploration of different problems. For instance, by implementing a random recommendation algorithm, researchers can generate uniform data that facilitates the study of unbiased recommendation. Furthermore, researchers can examine the effectiveness of different models by initiating two parallel simulators, and compare the

¹https://en.wikipedia.org/wiki/Long_tail

²https://en.wikipedia.org/wiki/Pareto_distribution

³<https://github.com/RUCAIBox/RecSysDatasets>

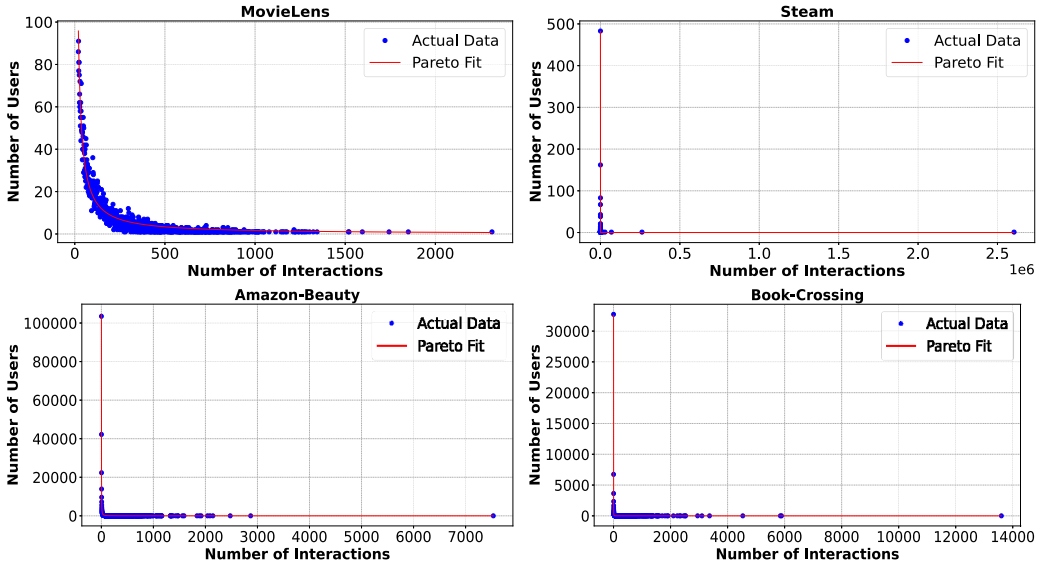


Fig. 4. The results of using $p(x)$ to fit real-world datasets. The blue points are the real-world data, and the red lines are the fitted distributions.

user feedback or actively ask user feelings about the recommended items. Beyond manually implementing the recommendation algorithms, one can also easily incorporate external recommendation library (e.g., RecBole⁴) into our simulator.

3.2.3 Role Playing. Beyond running our simulator in a complete virtual manner, we also allow real humans to involve into the simulation process. Specifically, a real human can act as an agent and interact with the recommender system or other agents. The human-controlled agent can browse the recommender system, search for movies, click on movies, and engage in activities like chatting or posting on social media, just like the other agents. This function actually introduces a novel execution fashion of our simulator, that is, running based on humanagent collaboration, which can help to improve the simulation effectiveness. For example, one can play as an agent, and actively correct the erroneous and hallucinatory behaviors of the other agents by chatting with them or posting one-to-many messages, which can make the simulated behaviors more realistic and reliable. If we regard human behaviors as “complete real,” and agent behaviors as “complete virtual,” human-agent collaboration enables us to explore more intermediate states between these boundary cases. Human behaviors are more accurate, but costly. Agent behaviors are inexpensive, but could be erroneous and hallucinatory. Human-agent collaboration can help to achieve better trade-offs between the human and agent data, enabling the utilization of their respective advantages.

3.2.4 System Intervention. To flexibly control the simulation process, our simulator can be intervened from outside in a global manner. In specific, one can “interview” the agents by asking them questions like “What would you say when you want to discuss the movies you’ve recently watched with others?” Additionally, people can also pause the simulator at any point and modify certain factors (e.g., user profiles) before resuming the simulation. This feature allows us to study many problems like the effects of emergency events or counterfactual user behaviors.

⁴<https://recbole.io/>

4 Experiments

To evaluate the effectiveness of our simulator, this section conducts extensive experiments from both agent and environment perspectives. Specifically, we aim to answer the following research questions:

- *RQ1*: How believable are the simulated user behaviors in comparison to real human behaviors, particularly within a recommender system and during chatting/broadcasting interactions?
- *RQ2*: How effective is the agent memory mechanism in producing rational and human-like behaviors?
- *RQ3*: How efficient is the simulator in terms of time cost, especially as the number of agents, API keys, and simulation epochs increase?
- *RQ4*: Can external interventions effectively influence agent behaviors as intended?

4.1 Agent-level Evaluation

4.1.1 Believability of the Simulated User Behaviors. To begin with, we evaluate the believability of the simulated behaviors, where we measure the behaviors in the recommender system and chatting/broadcasting behaviors separately.

Behaviors in the Recommender System. To evaluate the believability of the behaviors in a recommender system, we first sample 20 users as well as their interactions from a well-known recommendation dataset—Movielens-1M.⁵ Each user is assigned an age, gender, and occupation based on the dataset, while other profile attributes are left unassigned. Then, for each user, we leave her last a items as the evaluation ground truths, and use all the other items to initialize the agent profiles. For evaluation, we combine the ground truths with b negative items to form a recommendation list containing $(a + b)$ items. We present the recommendation list to the agent, and let it select a items to compare with the ground truth. Suppose for a user u , the ground truth and selected item sets are T_u and S_u , respectively, then we evaluate different models using the Overlap Ratio metric as follows:

$$p = \sum_{u \in U} \frac{|S_u \cap T_u|}{|T_u|},$$

where U is the set of all users, and larger p indicates better performance. The Overlap Ratio measures how well the agent's selected items align with the user's ground truth, effectively capturing the behavioral consistency between them and reflecting the believability of the agent in simulating user behavior. In the experiments, we set a and b with various values to verify the generality of the agent capability. In intuitively, selecting a smaller number of ground truth from a larger pool of candidates should be more difficult.

Different from traditional recommender models, which aims to generate recommendation lists for users, RecAgent is basically a user model, which targets at making selections from given recommendation lists. As a result, we compare RecAgent with the following models:

- *Embedding* [66]: in this method, for each user, we construct the user model by leveraging the average embedding of the items they have interacted with. To make selections, we compute the inner product between the user model and candidate item embeddings. The item embeddings are derived based on the movie descriptions using Sentence-BERT [67].
- *BERT* [29]: Inspired by previous work, we concatenate user profile and item information as context and transform it into a representation $h^{u,i}$ with Bert [68]. With the representation, we

⁵<https://grouplens.org/datasets/movielens/1m/>

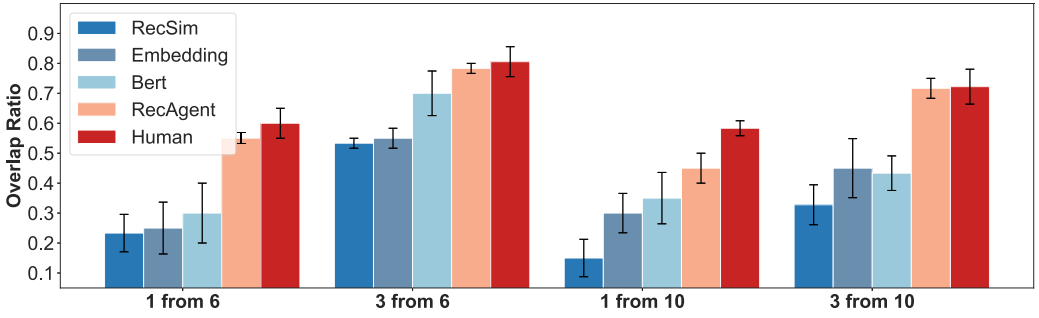


Fig. 5. Evaluation on the recommendation behaviors based on different (a, b) 's (discrimination capability).

train the user model to predict user satisfaction as $P(s|u, i) = \text{softmax}(\text{MLP}(h^{u,i}))$. The user selects items based on satisfaction scores.

- *RecSim* [26]: This is a well-known recommender simulator developed by Google Research. In this method, we first initialize the user vectors in a similar manner as the above model's, and then run the toolkit with default settings.
- *Real Human*: in this method, we present the user previous interactions and the current movie description to real humans, and let them make selections from the recommendation list. To minimize variability, we recruit three annotators and average their performance results.

The experiment results are presented in Figure 5, from which we can see: For the different (a, b) settings, the Embedding method performs similarly to RecSim, while the Bert method consistently outperforms both Embedding and RecSim across most configurations. By leveraging LLMs, our RecAgent simulator significantly improves performance compared to other methods. These findings highlight the potential of using LLM-based agents to simulate user behaviors. When comparing our simulator's results with real human performance, the gap is relatively small. On average, our simulator surpasses the best baseline by approximately 45.8%, and its performance is only 8.7% lower than that of real human evaluations. These results demonstrate that our method can predict items that users may prefer with greater accuracy, which reflects one aspect of believable user behavior. However, while this prediction capability is encouraging, it does not fully capture the overall believability of the agent's actions. To further explore this, we conducted additional experiments focused on the agents' ability to generate realistic and coherent user behavior sequences.

In addition to measuring the agents' discrimination capabilities, we also evaluated their potential to directly generate believable user behavior sequences (i.e., generation capability). To this end, we follow the above experiment settings to profile the agents with several early interactions and regard the following N items as **reference behaviors (RB)**. A straightforward evaluation method can be computing the overlap between the simulated and reference behaviors. However, in practice, a user may have multiple possible behaviors at each decision step, but only one of them is finally executed and logged into the system. This makes the reference behaviors not the only ground truth, and the simulated behaviors can also be reasonable even without any overlap with the reference behaviors. To overcome this challenge, we follow the previous work [69, 70] to conduct adversarial subjective evaluation, where a more believable simulator should be able to more readily "fool" the human annotator. In specific, we recruit three human annotators to make comparisons on RecAgent v.s. RB and RecSim v.s. RB. For each pair of comparison, the annotators are given: (i) the previously interacted movies of a user and (ii) two behavior sequences of the same user generated from different methods. The annotators are required to answer the following question: *According to the user's past behaviors, indicate which behavior sequence is more like the real human's?* They

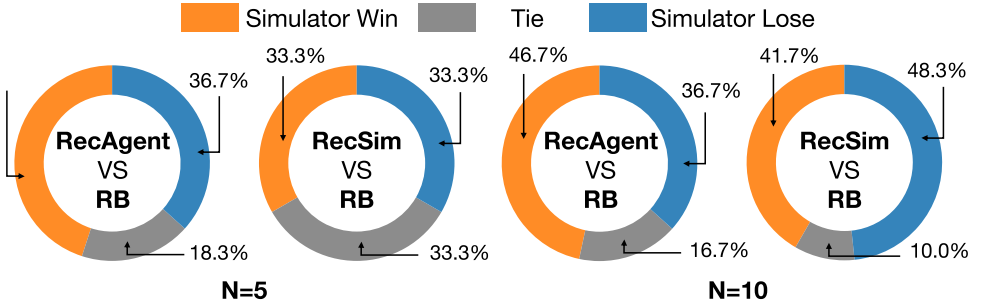


Fig. 6. Evaluation on the recommendation behaviors based on different N 's (generation capability).

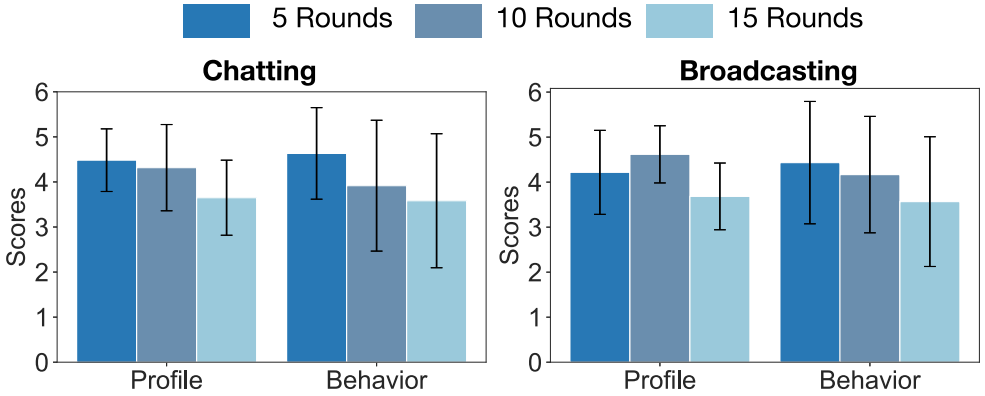


Fig. 7. Evaluation on the chatting and broadcasting behaviors.

can either choose one sequence or answer “indistinguishable.” If the annotators select results from RecAgent or RecSim, then the simulator win, otherwise, the simulator lose. The win, lose and indistinguishable rates of different simulators are reported in Figure 6. We can see, when $N = 5$, the win rate of RecAgent (45.0%) is significantly better than that of RecSim (33.3%), which suggest that our simulator can produce more believable user behavior sequences comparing with RecSim. When the generated sequence becomes longer, RecAgent can still outperform RecSim by about 5.0%, which demonstrates that the superiority of RecAgent is a general conclusion.

Chatting and Broadcasting Behaviors. In practice, it is hard to find datasets which simultaneously containing user recommendation, chatting, and broadcasting behaviors. For traditional simulation strategies, the user model trained on one type of user behaviors can hardly generalize to the other ones, since the semantic spaces and patterns of different behaviors may have significant gaps. For example, the above recommendation simulator Embedding or RecSim cannot generate user chatting or broadcasting behaviors. For our method, since LLMs have already learned almost all the web corpus, they have sufficient knowledge to understand and simulate different behaviors. In this section, we evaluate whether our simulator can produce believable user behaviors even without fine-tuning on domain-specific datasets. In specific, we deploy 20 agents in the simulator, and observe their chatting and broadcasting behaviors after 5, 10, and 15 rounds of executions. We recruit three annotators, and present them with the agents’ profiles and previously watched movies. They are required to answer the following questions: (1) *Whether the chatting behaviors of the agent are believable according to its profiles?* (2) *Whether the chatting behaviors of the agent are believable according to its previous behaviors?* (3) *Whether the broadcasting behaviors of the*

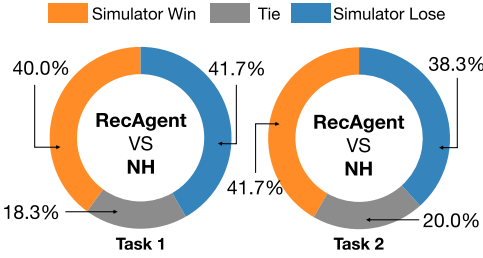


Fig. 8. Evaluation on different key functions of the memory module.

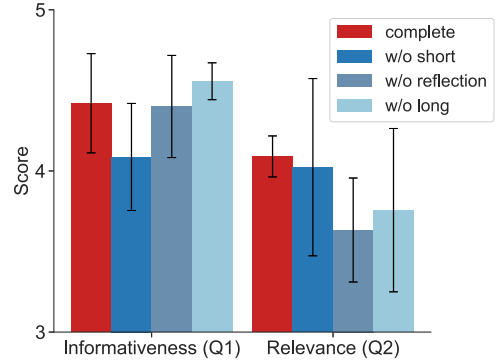


Fig. 9. Evaluation on the memory module as a whole.

agent are believable according to its profiles? (4) Whether the broadcasting behaviors of the agent are believable according to its previous behaviors? The annotators have to give a score between 1 and 5 (higher rating indicates more believable) to each of the above questions. The results are reported by averaging all the scores across the annotators and agents. From Figure 7, we can see, most of the results are higher than 4, suggesting that our simulator can indeed produce believable chatting and broadcasting behaviors. However, as the simulator runs longer, the performances goes down in general. After 15 rounds of execution, the believability scores for all the questions are lower than 4. We speculate that, after 15 rounds, the agent memory may have stored a large amount of contents, and the LLM may lose its attention and fail to extract valuable information to simulate behaviors.

4.1.2 Believability of the Agent Memory Mechanism. In this section, we evaluate the rationality of the agent memory mechanism, which is the key part for driving user behaviors. To begin with, we evaluate the believability of the key functions of the memory module, focusing on two tasks: (1) Summarizing given short-term memories to be transferred to the long-term memory, and (2) generating reflections based on given long-term memories. We first let RecAgent and **non-expert humans (NH)** to perform each of the above tasks, and then recruit three annotators to label which result is more like the real human's, if they are unsure, they can answer "indistinguishable." For each task, we generate 20 pairs of samples for comparison. The results are presented in Figure 8, from which we can see: for the first task, about 40% annotations believe that RecAgent is more like humans, which is only 1.7% lower than NH. For the second task, the support ratio of RecAgent slightly surpasses RH by about 3.3%. These observations demonstrate the believability of the agent memory working mechanisms.

Beyond the above experiments on evaluating different parts of the memory module independently, we also evaluate whether the memory mechanism as a whole can produce informative and relevant knowledge to support user behaviors. To see the advantages of our memory module, we compare it with its three variants: (1) in w/o short, we remove the short-term memory, and the final extracted information is only from the long-term memory, (2) in w/o long, we remove the long-term memory, and the final extracted information is only from the short-term memory, and (3) in w/o reflection, we inactivate the reflection mechanism in long-term memory. We randomly sample 15 agent behaviors like movie selection, friend chatting and posting message, and use them as testing cases. For each case, we recruit three human annotators to evaluate the extracted information from different variants. We ask each annotator to answer the following two questions: (1) *For the current behavior, whether the extracted memories are informative?* (2) *For the current behavior, whether the extracted memories are relevant?* Each annotator is required to give a score from 1 to 5 for each

question. Then, we average the scores across all the annotators and testing cases for each variant. The results are presented in Figure 9, from which we can see: if we remove the short-term memory, the informativeness score is severely lowered. We speculate that the short-term memory may have stored a lot of diverse user recent memories, which is important for improving the informativeness. On the other hand, since the long-term memory in our simulator is extracted based on embedding similarities, the obtained information can be highly relevant with the current user behavior. As a result, we can see, if we remove the long-term memory or the reflection information, the relevance scores are lowered. By combining all the memory components, our complete memory module can achieve the best relevance score, and a comparable informativeness score to the best one.

4.2 Environment-level Evaluation

In addition to the above experiments on agent-level evaluation, in this section, we evaluate the designed environment.

4.2.1 Efficiency Analysis. To begin with, we analyze the efficiency of our simulator to provide more insights. In specific, we focus on the following four problems:

- *P1*: How does the time cost increase as the number of agents become larger in each epoch?
- *P2*: How does the time cost increase as the number of API keys become larger in each epoch?
- *P3*: How does the time cost increase as the number epochs become larger?
- *P4*: What are the time costs of different agent behaviors?

For the first problem, we fix the number of API keys as 1, and observe the time cost of one round execution of our simulator when the number of agents ranges from 1 to 500. In our simulator, we support parallel API key invocation. For the second problem, we fix the number of agents as 10, and observe the time cost of one round execution when the number of API keys ranges from 1 to 5. In the third problem, we observe the total simulation cost as the number of epochs becomes larger, where we fix the number of API keys and agents as 1 and 10, respectively. In the fourth problem, we analyze the time costs of different agent behaviors, where the settings on API keys and agents follow the above experiment, and we focus on three types of behaviors including “movie selection,” “friend chatting,” and “message posting.” In this experiment, we run our simulator on a server equipped with six CPUs (Intel(R) Xeon(R) Gold 6148 CPU @ 2.40GHz) and 4 GB of memory.

Result Analysis. The results are presented in Figure 10. We can see, when the number of API keys is fixed, the time cost increases as the number of agent becomes larger. For example, when we deploy 10 agents, the time cost is about 220 s for each round of simulation, while when we have 100 agents, the simulation time becomes 1.9 hours under the same setting. It should be noted that, in this experiment, we make all agents take actions. In practice, the number of active agents can be much smaller as discussed in Section 3.2.1, which may greatly lower the time cost. By incorporating more API keys, the time cost can be lowered, which demonstrates the effectiveness of our designed mechanism for parallel API key invocation. However, we find that the results fluctuate a lot, and the variance is high, which is perhaps due to the unstable network speeds. As the number of rounds becomes larger, the time cost rises with an increasing acceleration rate, possibly because the information gradually accumulates as we run the simulator, which may lead to longer processing time. Among different types of behaviors, “friend chatting” costs most, since it needs to generate more complex contents. In practice, we found that 10 agents cost about 0.25 dollars per round based on ChatGPT, and the cost increases almost linearly as the number of agent becomes larger.

4.2.2 Case Studies on System Intervention. As discussed in Section 3.2.4, our simulator allows external interventions. In this section, we evaluate whether such interventions can effectively change agent behaviors as expected. In specific, we change the agents’ profile information, and

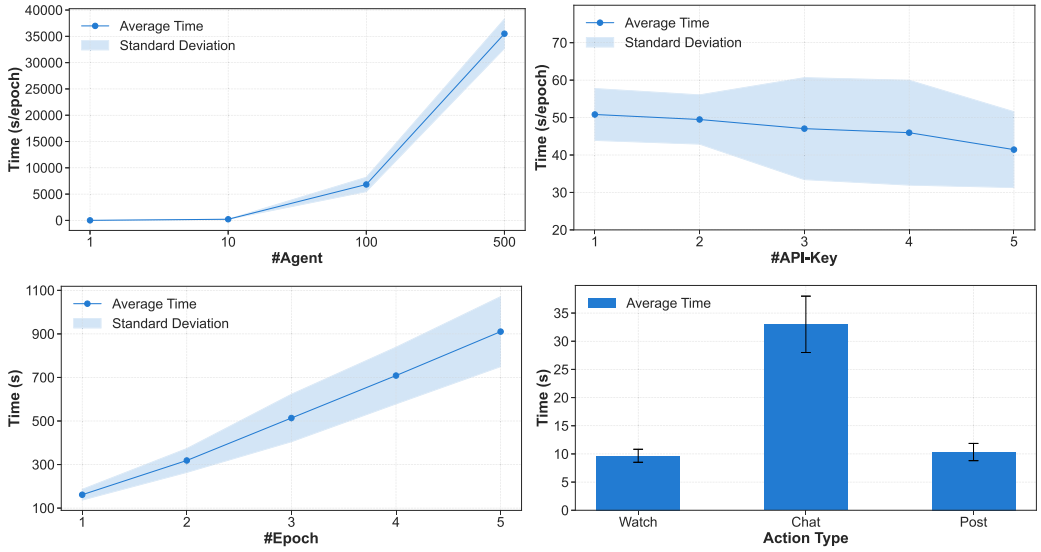


Fig. 10. The results of efficiency analysis of our simulator.

observe how their behaviors can be influenced. In the experiments, we first run our simulator for five rounds, and then separate the simulation into two branches: in the first branch, we change the agent profile information (*Intervention Branch*), while in the second branch, we do not make any intervention (*Original Branch*). For each branch, we continue to run our simulator for five rounds. Finally, we compare the differences between these two branches in the scenarios of friend chatting.

We present the results in Figure 11. In the left case, we change the traits and interests of an agent called David Smith. Originally, David Smith likes sci-fi movies. Thus, we may find sentences like “I did watch this amazing sci-fi thriller called <Inception>. It’s mind-blowing!” or “I’m more into sci-fi, thriller, and suspense movies. They always keep me on the edge of my seat.” in his dialogs. However, if we change his preference to be a lover of family-friendly movies, then we observe sentences like “By the way, have you come across any good family-friendly or romantic movies? I’m in the mood for something heartwarming.” In the right case, we can find similar results. After introducing sci-fi, thriller, and action movies into the agent’s preference, it says “I’ve always loved the thrill and excitement of sci-fi, thriller, and action movies. They really get my adrenaline pumping.”

In addition to compare the chatting behaviors before and after the intervention, we also actively interview the agents to let it recommend a movie to the others, and explain the reasons. Ideally, the agent should make recommendations according to its own preference, and if its preference has been changed, the recommended movies should be accordingly altered. We interview the same agents as the above experiment. The results are presented in Figure 12. For both cases, we can see, the agent can successfully change its recommended movies according to its preference. In addition, the recommendation reasons suggest that the recommendation changes are exactly because of the preference differences before and after the intervention.

5 Potential Applications of Our Simulator

In the above sections, we demonstrate the believability of our simulator. We argue that based on such believability, we actually can provide an affordable, resettable, and intervenable platform for user behavior studies. In this section, we show the potential of our simulator on this point.

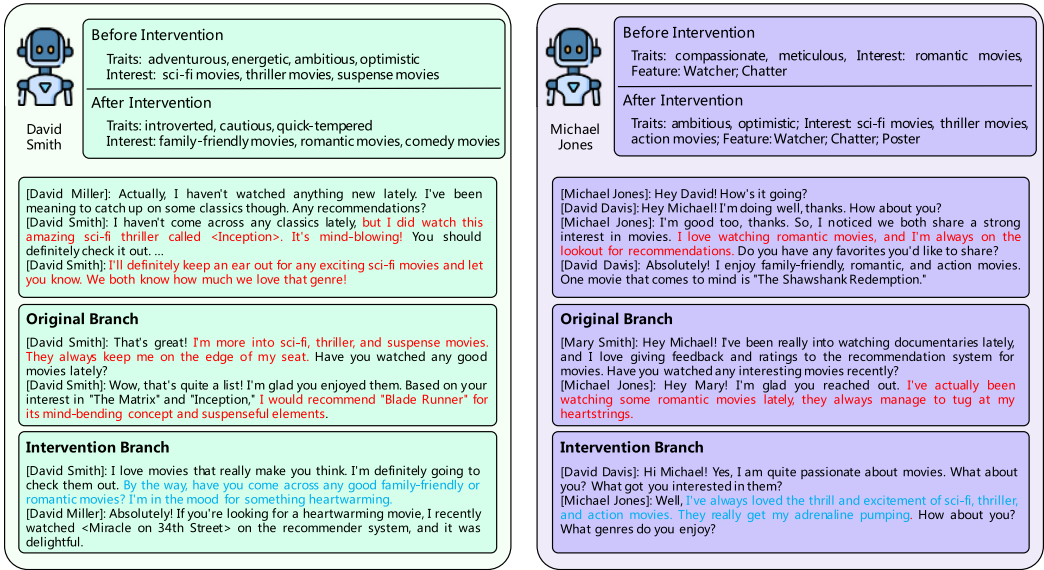


Fig. 11. Case studies on system intervention in the scenarios of friend chatting. The gray-colored region indicates the dialogs of the agent discovered in the first five round execution.

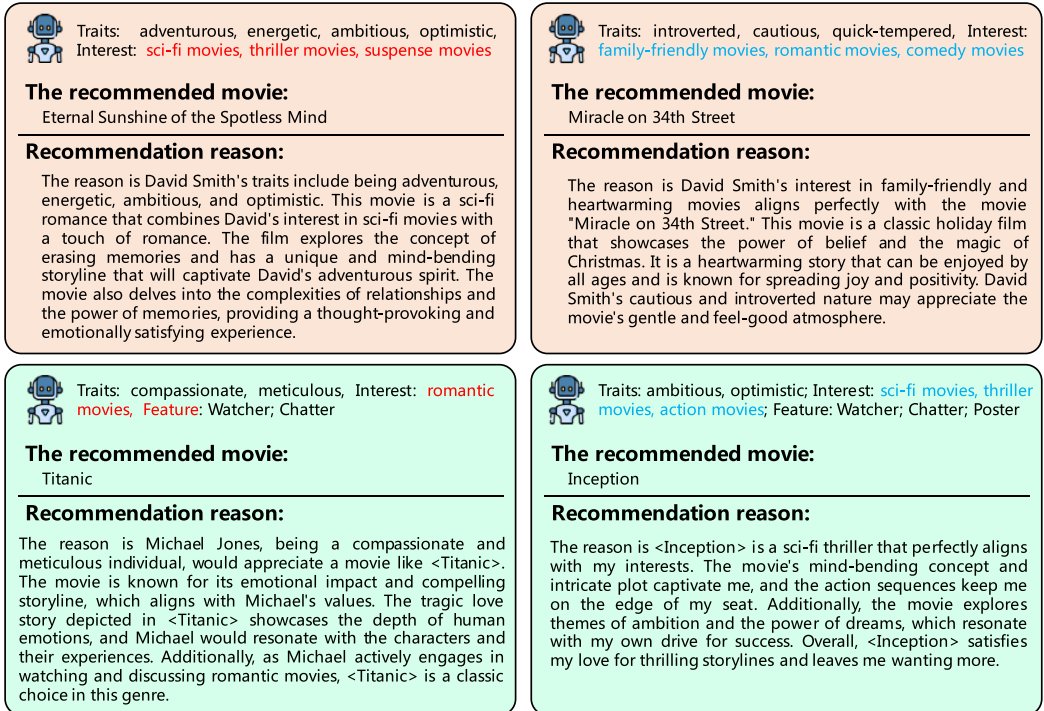


Fig. 12. Case studies on system intervention by actively interviewing the agents.

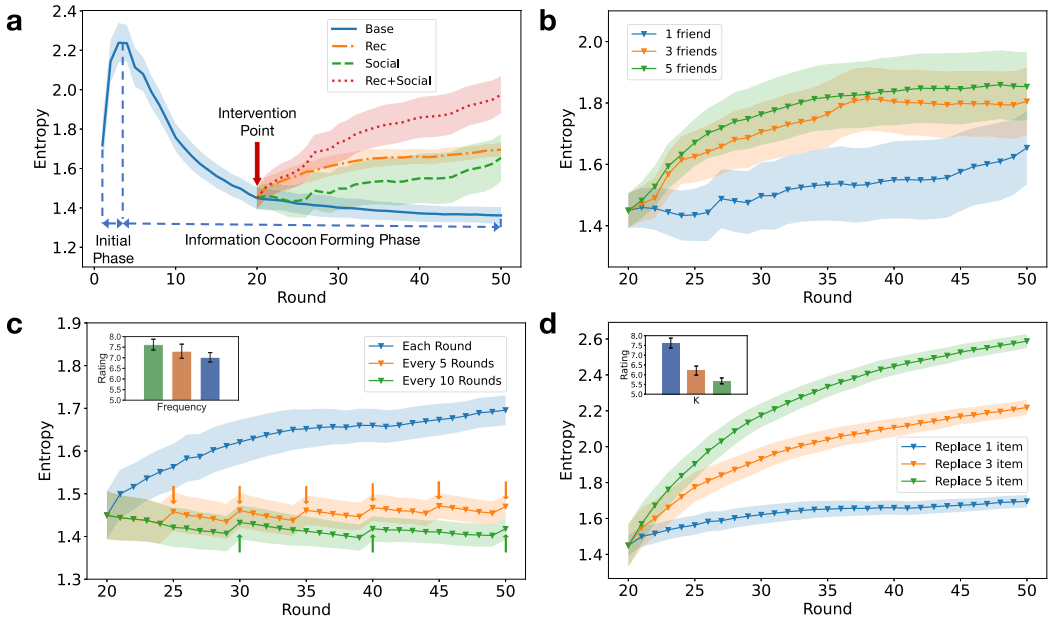


Fig. 13. *Studying the information cocoon phenomenon with RecAgent.* (a) The results of reproducing information cocoon phenomenon and the effects of two potential strategies (i.e., Soc-Strategy and Rec-Strategy) for alleviating this problem. (b) Detailed studies on the Soc-Strategy, where we add N ($N = 1, 3, 5$) friends with different interests to each user. (c) Detailed studies on the Rec-Strategy, where we incorporate randomness to the recommendation results every 1, 5, and 10 rounds. (d) Detailed studies on the Rec-Strategy, where we replace N ($N = 1, 3, 5$) items in the recommendation list.

5.1 Studying the Information Cocoon Phenomenon with Our Simulator

Information cocoon refers to the problem that users may only access information similar to their own preference, but lose the opportunity to view more diverse messages [71]. In this section, we simulate the phenomenon of information cocoons based on our simulator, and try to propose several potential strategies to alleviate this problem. In specific, we deploy 50 agents, allowing them to freely search movies, chat with each other and broadcast messages. We use matrix factorization as the recommendation algorithm, which generate five items each time. The model is incrementally trained after each round of execution, and the agent-item interactions for model training come from the agent searching behaviors and system recommendations. We use entropy to measure the information cocoon phenomenon [72], which is computed as follows:

$$E = -\frac{1}{|U|} \sum_{u \in U} \sum_{c \in C} f_{u,c} \log f_{u,c},$$

where U and C are the sets of all users and item categories. $f_{u,c}$ is the frequency of category c recommended by the system to user u . Smaller entropy indicates more severe information cocoon.

We run our simulator by 50 rounds, and report the entropy of the recommended items in Figure 13(a) (the blue curve). Initially, the entropy increases as new users receive a broad range of recommendations, but after about five rounds, it begins to decline, with the final entropy reduced by approximately 8.5% from its peak. This reflects how the recommendation system, through continued user interaction, gradually tailors its suggestions to user preferences, narrowing the diversity of recommended items and leading to the formation of an information cocoon, consistent with previous analyzes [72, 73].

To alleviate the information cocoon problem, we study two strategies: (1) improving the randomness of the recommendation algorithm, and (2) increasing social relations of the users. For the first strategy, we replace one of the five recommended items with a random one at each round (*Rec-Strategy*). This approach is supported by Piao et al. [72], who show that introducing randomness into recommendations can counteract the formation of information cocoons by encouraging users to explore new content. For the second strategy, we add one friend with different interests to each user (*Soc-Strategy*). As Hou et al. [74] highlight, social networks tend to reinforce information cocoons due to users interacting within their existing like-minded circles. Therefore, to mitigate this, we incorporate social relationships with agents who have different interests, aiming to introduce more diverse information. We apply these strategies after 20 round of execution, and report their effectiveness in Figure 13(a) by the orange and green curves. We can see, both of these strategies can improve the entropy, and the Rec-Strategy is more significant. By combining these strategies (see the red curve of Figure 13(a)), we can observe further improvement on alleviating the information cocoon problem.

In the next, we conducted detailed experiments on the Rec- and Soc-Strategies separately. For the Rec-strategy, we first study the intervention frequency of the recommendation algorithm. Specifically, we introduce randomness into the recommendation results every 1, 5, and 10 rounds. From Figure 13(c), we observe that as the frequency of intervention increases, the information cocoon problem is better alleviated. In particular, intervening every round increases entropy by approximately 15.3% and 19.6% compared to interventions every 5 rounds and 10 rounds, respectively. The more frequently users are exposed to randomly recommended content, the more diverse the information they encounter, which helps mitigate the formation of information cocoons [72]. However, more frequently incorporating randomness may lower user satisfaction. To verify such intuition, we ask each agent to give a score in [1,10] to reflect its satisfaction with the recommendation qualities. From the upper-left subfigure of Figure 13(c), we can see, as the intervention frequency becomes higher, user satisfaction is indeed lowered. This observation suggests that while actively incorporating randomness into the recommendation algorithm is a direct and effective strategy for alleviating the information cocoon problem, it may also hurt user experiences and lead to lowered user retention rates. One should carefully consider such a trade-off when using the Rec-strategy.

Then, we study the influence of the number of replaced items (N) in the recommendation list. In specific, we replace 1, 3, and 5 items with random ones, and intervene the recommender system every round. From the results presented in Figure 13(d), we can see, replacing more items can lead to better effect on alleviating the information cocoon problem. In specific, when $N = 3$ and 5, the entropy can be improved by about 30.8% and 52.6%, respectively, as compared with $N = 1$. Similar to the above experiments, replacing more items may also lower user satisfaction, which can be evidenced from the upper-left subfigure of Figure 13(d).

For the Soc-Strategy, we study the influence of the number of added social relations to each agent in the Soc-Strategy. From the results presented in Figure 13(b), we can see, by adding more friends, the information cocoon problems can be better alleviated. This is consistent with the findings of Wu [75] and Kong et al. [76], which suggest that an increase in social connections can diversify the information users encounter, thereby weakening the impact of the information cocoon. In our experiment, we observe that as the number of friends increases, the recommendation system introduces more diverse items, which helps to break the information cocoon.

5.2 Studying the User Conformity Phenomenon with Our Simulator

User conformity behaviors mean that the users may easily follow their friends' attitudes, leading to biased social opinions and beliefs. In this section, we simulate user conformity behaviors using our

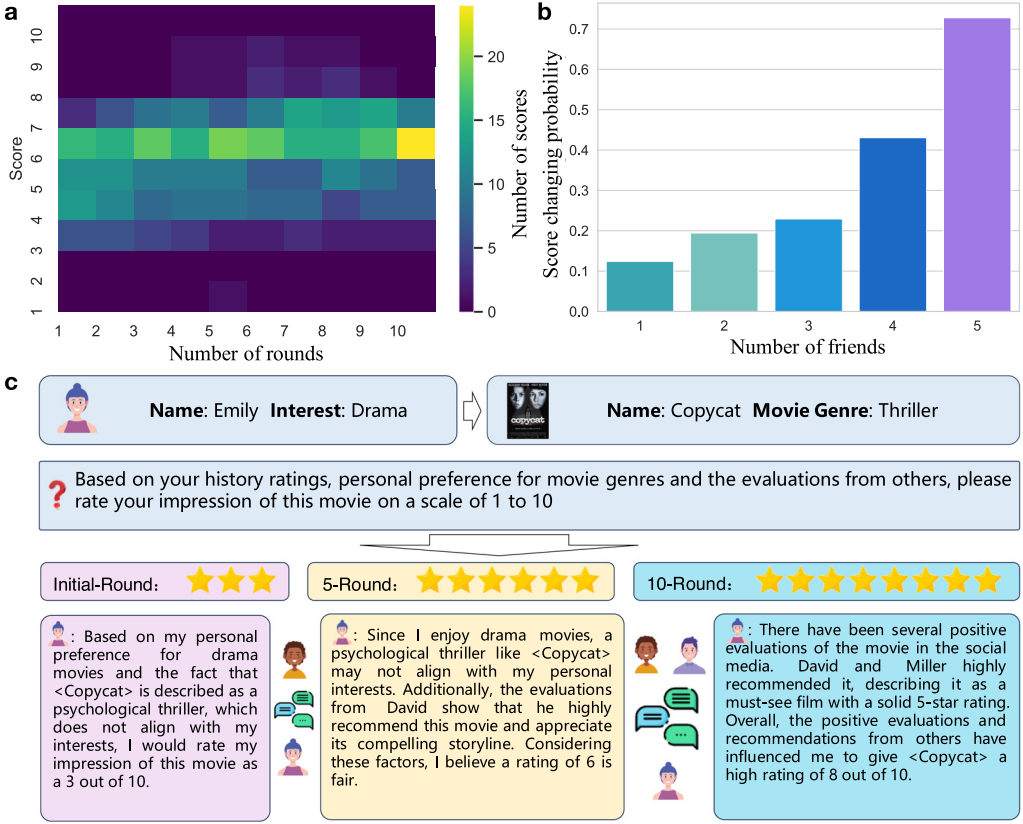


Fig. 14. *Studying the user conformity phenomenon with RecAgent.* (a) The change of the score distributions with different rounds of execution. (b) The relation between the number of friends of the agents and their score changing probability. (c) A toy example illustrating the score changing process of an agent as well as the reasons.

simulator, and analyze their key characters. In specific, we first let five agents with sufficient social connections to watch a movie, and then they may share opinions with their friends *via* one-to-one chatting or one-to-many broadcasting. As the simulation goes on, more and more opinions are accumulated and diffuse in the social network, continually influencing agent attitudes toward the movie. After each system round, we ask each agent to score the movie in [1,10]. In Figure 14(a), we present the score distribution across different rounds. Initially, the scores are more evenly distributed in the range of 3–8, but they eventually concentrate around 6 and 7. These results indicate that conformity bias in recommender systems may be driven by social influence, as users tend to align their behavior with that of their friends. This finding aligns with previous studies that highlight the role of social trust and influence in shaping user preferences. Ma et al. [77] demonstrated that social trust encourages users to adopt their friends' preferences, while Chaney et al. [78] showed how social connections influence item adoption probabilities in personalized recommendations. Tang et al. [79] further highlighted the role of trust in shaping user behaviors within connected environments. These findings support our observation that social relationships can drive conformity behavior, leading users to align their preferences with those of their social circles.

In the next, we present the relation between the friend number of an agent and the probability of attitude change in Figure 14(b). The attitude change is measured based on the following equation:

$$AC(i) = \frac{1}{10|U_i|} \sum_{u \in U_i} \sum_{r \in [1,10]} 1(s_{u,r-1} \neq s_{u,r}),$$

where U_i is the set of agents which have i friends. r is the index of the system round. $s_{u,r}$ is the score of user u at round r . $1(s_{u,r-1} \neq s_{u,r})$ is an indicator function, which is 1, if the score is changed between round $r - 1$ and r , and 0, otherwise. We observe that agents with more friends are more likely to change their scores and exhibit conformity behavior. This finding aligns with Tang et al. [80], who demonstrated that a greater number of close social connections increases the likelihood of conformity within large social networks. In Figure 14(c), we present a specific example, illustrating the attitude changes of the agent before and after accessing its friend's opinions. These results suggest that the agents are indeed influenced by their social relations, which promotes the formation of conformity behaviors.

6 Discussion

To further explore the characteristics and applications of RecAgent, we present the following discussion.

Relation with the Previous Work. RecAgent is inspired by pioneering work on Generative Agents [59], sharing the same fundamental idea of leveraging LLMs for human simulation. However, we focus on specific web user behaviors related with recommendation and social network. This narrower scope simplifies the simulation process while enhancing its controllability. Web applications such as recommendation and social network are playing pivotal roles in modern business. We believe simulating user behaviors within these applications may have more significant impacts.

Extensibility of RecAgent. RecAgent is designed to be a highly extensible framework. From the agent perspective, the agent modules are fully decomposed, allowing for easy modification or removal without impacting other modules. This modularity not only enables the implementation of more complex user behaviors by adding new modules but also allows for seamless adaptability across various domains. Here, a potential challenge lies in the prompting method. After revising the agent modules, it may be necessary to adjust the prompting strategy accordingly. From the simulator perspective, the framework provides flexibility to easily design additional user behaviors based on the paradigms employed for existing recommendation and social interaction behaviors. Beyond recommender systems, RecAgent's flexibility extends to fields like search engines and conversational systems. In search engines, it can model user preferences and behaviors to generate realistic queries, helping to optimize search algorithms. In conversational systems, it can replicate dialogue flows and user engagement, facilitating the evaluation of conversation strategies and enhancing personalized response mechanisms. This versatility makes RecAgent a powerful simulator for dynamic, context-aware applications across diverse domains.

Potential Impacts of RecAgent. Real-data and simulation-based studies are two common research paradigms in the field of user behavior analysis. Real-data-based studies can accurately learn the user real preferences and behavior patterns. However, this paradigm confronts two salient challenges. Firstly, this paradigm proves resource-intensive and lacks sustainability. This limitation restricts its utility to a narrow band of fixed problems, thereby impeding swift adaptation to burgeoning new problems in Web 2.0 (e.g., RL-based recommendation and explainable recommendation [81–85]). Second, the richness of user interaction data in real-world contexts can be difficult to capture comprehensively. For instance, a user might choose to watch a film based on a friend's casual mention in conversation, an influencing factor that is difficult to capture through the lens of a

recommender system. These hurdles significantly impact the trajectory of our in-depth investigation into recommender systems.

Simulation based studies can help explore a wider range of application scenarios. However, the majority of user behavior related problems are highly subjective, and simulating the human mind can be challenging. Although there have been several promising user simulators, such as RecSim⁶ and VirtualTaobao,⁷ they often simulate human minds using simple and naive functions, which may not be powerful enough to handle the complexity of the user decision-making mechanism.

RecAgent opens a novel direction for advancing simulation-based studies by leveraging LLMs to capture human subjective preferences. As LLMs continue to develop, RecAgent can be further enhanced as we provide a flexible interface to incorporate any LLM.

Limitations. This work may have the following limitations, which we plan to address in future:

- *Discretization of Real-world Continuous Time:* Our simulator operates on a round-by-round basis. Although time information is considered, it is discretized, which deviates from real-world scenarios. This approximation limits users from taking actions between rounds, resulting in less flexibility compared to real-world situations.
- *Incomplete Simulation of User Decision Factors:* RecAgent focuses on simulating user behaviors within recommender systems and social media. However, we may overlook various factors that also influence user decisions. For instance, a father might choose to watch cartoons with his son, even if they are not of personal interest to him.
- *LLMs Not Tailored for User Behavior Analysis:* Our simulator does not involve fine-tuning LLMs to enhance their adaptability to recommendation tasks. To achieve satisfactory simulation performance, we carefully crafted prompts for various user behaviors and implemented post-processing and error-tolerance mechanisms. However, these prompts may not be robust across different LLMs, and the LLMs might not always follow instructions consistently. Ongoing research on enabling structured outputs from LLMs [86] is something we plan to incorporate in future work.
- *Challenges in Evaluation:* Subjective evaluation is a significant challenge and an open problem in the field of simulation. While we employed both human annotations and comparisons with real human data to assess the realism of simulated user behaviors, this approach may still miss some subtle nuances of real-world interactions. The inherent subjectivity in human evaluations and potential biases in the datasets used could influence the outcomes. In future work, we aim to address these challenges by using more diverse datasets and integrating additional objective metrics to enhance the robustness of our evaluations.
- *Security and Privacy Considerations:* While our simulation environment is designed to mimic user behaviors realistically, the inclusion of detailed profiling and behavioral data raises potential security and privacy concerns. The data we use is either generated by LLMs or obtained from publicly available datasets. Currently, the security of our system heavily relies on the safeguards provided by the LLMs themselves, which have undergone RLHF to mitigate harmful outputs. However, there remains a risk of data breaches or misuse, particularly if the generated or collected data is improperly handled. In future work, we plan to enhance the security protocols of our system, ensuring that data is securely stored, processed, and accessed, and to explore privacy-preserving techniques to minimize the risks associated with the use of personal data in simulations.

⁶<https://github.com/google-research/recsim>

⁷<https://github.com/eyounx/VirtualTaobao>

We believe this is a fundamental direction, which can shift the paradigm of recommendation studies from methods dominated by real-data to strategies combining real-data and simulation. Furthermore, user behavior analysis is just a sub-field of human-centered AI, and user simulation should be a pervasive challenge across all tasks in this domain. We believe RecAgent can serve as an example to inspire researchers tackling other subjective simulation problems in human-centered AI.

7 Conclusion and Future Work

This paper introduces a novel direction of user behavior simulation with LLM-based agents. To implement this idea, we propose an effective agent framework, and carefully design its profile, memory and action modules. In addition, we also design a multi-agent environment, where different agents can freely take actions and evolve themselves. Based on extensive experiments, we demonstrate the believability of our simulator. To show the potential of our simulator, we use it to study the phenomena of information cocoons and user conformity behaviors. This paper opens the door of user behavior simulation with LLM-based agents. There is much room left for improvement. For instance, one can fine-tune existing LLMs to enhance their capabilities for following instructions within the field of user behavior analysis. In addition, people can also extend RecAgent to other human-centered AI tasks, such as information retrieval and so on.

References

- [1] Saeid Nahavandi. 2019. Industry 5.0—a human-centric solution. *Sustainability* 11 (08 2019), 4371. DOI : <https://doi.org/10.3390/su11164371>
- [2] Elizaveta Stavinova, Alexander Grigorievskiy, Anna Volodkevich, Petr Chunaev, Klavdiya Bochenina, and Dmitry Bugaychenko. 2022. Synthetic data-based simulators for recommender systems: A survey. arXiv:2206.11338. Retrieved from <https://arxiv.org/abs/2206.11338>
- [3] Krisztian Balog and ChengXiang Zhai. 2024. User simulation for evaluating information access systems. *Foundations and Trends in Information Retrieval* 18, 1–2 (2024), 1–261. DOI : <https://doi.org/10.1561/15000000098>
- [4] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. arXiv:2303.18223. Retrieved from <https://arxiv.org/abs/2303.18223>
- [5] Jesús Bobadilla, Fernando Ortega, Antonio Hernando, and Abraham Gutiérrez. 2013. Recommender systems survey. *Knowledge-based systems* 46 (2013), 109–132.
- [6] Paul Resnick and Hal R. Varian. 1997. Recommender systems. *Commun. ACM* 40, 3 (1997), 56–58.
- [7] Linyuan Lü, Matúš Medo, Chi Ho Yeung, Yi-Cheng Zhang, Zi-Ke Zhang, and Tao Zhou. 2012. Recommender systems. *Physics reports* 519, 1 (2012), 1–49.
- [8] Pasquale Lops, Marco De Gemmis, and Giovanni Semeraro. 2011. Content-based recommender systems: State of the art and trends. *Recommender systems handbook* (2011), 73–105.
- [9] Michael J. Pazzani and Daniel Billsus. 2007. Content-based recommendation systems. In *The Adaptive Web: Methods and Strategies of Web Personalization*. Springer, 325–341.
- [10] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the Web Conference*, 173–182.
- [11] Xiaoyuan Su and Taghi M. Khoshgoftaar. 2009. A survey of collaborative filtering techniques. *Advances in Artificial Intelligence* 2009 (2009), Article 421425, page 2. DOI : <https://doi.org/10.1155/2009/421425>
- [12] Zhi-Dan Zhao and Ming-Sheng Shang. 2010. User-based collaborative-filtering recommendation algorithms on hadoop. In *2010 3rd International Conference on Knowledge Discovery and Data mining*. IEEE, 478–481.
- [13] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *10th International Conference on World Wide Web*, 285–295.
- [14] Shoujin Wang, Liang Hu, Yan Wang, Longbing Cao, Quan Z. Sheng, and Mehmet Orgun. 2019. Sequential recommender systems: challenges, progress and prospects. arXiv:2001.04830. Retrieved from <https://arxiv.org/abs/2001.04830>
- [15] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. IEEE, 197–206.
- [16] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 974–983.

- [17] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and powering graph convolution network for recommendation. In *43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 639–648.
- [18] Yueming Sun and Yi Zhang. 2018. Conversational recommender system. In *41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, 235–244.
- [19] Konstantina Christakopoulou, Filip Radlinski, and Katja Hofmann. 2016. Towards conversational recommender systems. In *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 815–824.
- [20] Lucas Bernardi, Sakshi Batra, and Cintia Alicia Bruscantini. 2021. Simulations in recommender systems: An industry perspective. arXiv:2109.06723. Retrieved from <https://arxiv.org/abs/2109.06723>
- [21] Haruka Kiyohara, Kosuke Kawakami, and Yuta Saito. 2021. Accelerating offline reinforcement learning application in real-time bidding and recommendation: Potential use of simulation. arXiv:2109.08331. Retrieved from <https://arxiv.org/abs/2109.08331>
- [22] Jin Huang, Harrie Oosterhuis, Maarten De Rijke, and Herke Van Hoof. 2020. Keeping dataset biases out of the simulation: A debiased simulator for reinforcement learning based recommender systems. In *14th ACM Conference on Recommender Systems*, 190–199.
- [23] Stavros P. Adam, Stamatios-Aggelos N. Alexandropoulos, Panos M. Pardalos, and Michael N. Vrahatis. 2019. No free lunch theorem: A review. *Approximation and Optimization: Algorithms, Complexity and Applications* (2019), 57–82.
- [24] Vladimir Provalov, Elizaveta Stavinova, and Petr Chunaev. 2021. Synevarec: A framework for evaluating recommender systems on synthetic data classes. In *2021 International Conference on Data Mining Workshops (ICDMW)*. IEEE, 55–64.
- [25] Krisztian Balog, David Maxwell, Paul Thomas, and Shuo Zhang. 2022. Report on the 1st simulation for information retrieval workshop (Sim4IR 2021) at SIGIR 2021. In *ACM SIGIR Forum*, Vol. 55. ACM New York, NY, 1–16.
- [26] Eugene Ie, Chih-wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019. Recsim: A configurable simulation platform for recommender systems. arXiv:1909.04847. Retrieved from <https://arxiv.org/abs/1909.04847>
- [27] David Rohde, Stephen Bonner, Travis Dunlop, Flavian Vasile, and Alexandros Karatzoglou. 2018. Recogym: A reinforcement learning environment for the problem of product recommendation in online advertising. arXiv:1808.00720. Retrieved from <https://arxiv.org/abs/1808.00720>
- [28] Jing-Cheng Shi, Yang Yu, Qing Da, Shi-Yong Chen, and An-Xiang Zeng. 2019. Virtual-taobao: Virtualizing real-world online retail environment for reinforcement learning. In *AAAI Conference on Artificial Intelligence*, Vol. 33, 4902–4909.
- [29] Weiwei Sun, Shuo Zhang, Krisztian Balog, Zhaochun Ren, Pengjie Ren, Zhumin Chen, and Maarten de Rijke. 2021. Simulating user satisfaction for the evaluation of task-oriented dialogue systems. In *44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2499–2506.
- [30] Weiwei Sun, Shuyu Guo, Shuo Zhang, Pengjie Ren, Zhumin Chen, Maarten de Rijke, and Zhaochun Ren. 2023. Metaphorical user simulators for evaluating task-oriented dialogue systems. *ACM Transactions on Information Systems* 42, 1 (2023), 1–29.
- [31] Sirui Yao, Yoni Halpern, Nithum Thain, Xuezhi Wang, Kang Lee, Flavien Prost, Ed H. Chi, Jilin Chen, and Alex Beutel. 2021. Measuring recommender system effects with simulated users. arXiv:2101.04526. Retrieved from <https://arxiv.org/abs/2101.04526>
- [32] Behnam Rahdari, Peter Brusilovsky, and Branislav Kveton. 2024. Towards simulation-based evaluation of recommender systems with carousel interfaces. *ACM Transactions on Recommender Systems* 2, 1 (2024), 1–25.
- [33] Jingjing Zhang, Gediminas Adomavicius, Alok Gupta, and Wolfgang Ketter. 2020. Consumption and performance: Understanding longitudinal dynamics of recommender systems via an agent-based simulation framework. *Information Systems Research* 31, 1 (2020), 76–101.
- [34] Naieme Hazrati and Francesco Ricci. 2022. Recommender systems effect on the evolution of users’ choices distribution. *Information Processing & Management* 59, 1 (2022), 102766.
- [35] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. 2024. Recommender systems in the Era of large language models (LLMs). *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 6889–6907.
- [36] Danial Ebrat and Luis Rueda. 2024. Lusifer: LLM-based user simulated feedback environment for online recommender systems. arXiv:2405.13362. Retrieved from <https://arxiv.org/abs/2405.13362>
- [37] Feiran Huang, Zhenghang Yang, Junyi Jiang, Yuanchen Bei, Yijie Zhang, and Hao Chen. 2024. Large language model interaction simulator for cold-start item recommendation. arXiv:2402.09176. Retrieved from <https://arxiv.org/abs/2402.09176>

- [38] Xiaolei Wang, Xinyu Tang, Wayne Xin Zhao, Jingyuan Wang, and Ji-Rong Wen. 2023. Rethinking the evaluation for conversational recommendation in the era of large language models. arXiv:2305.13112. Retrieved from <https://arxiv.org/abs/2305.13112>
- [39] Se eun Yoon, Zhankui He, Jessica Maria Echterhoff, and Julian McAuley. 2024. Evaluating large language models as generative user simulators for conversational recommendation. arXiv:2403.09738 Retrieved from <https://arxiv.org/abs/2403.09738>
- [40] Lixi Zhu, Xiaowen Huang, and Jitao Sang. 2024. A LLM-based controllable, scalable, human-involved user simulator framework for conversational recommender systems. arXiv:2405.08035. Retrieved from <https://arxiv.org/abs/2405.08035>
- [41] Zhenduo Wang, Zhichao Xu, Qingyao Ai, and Vivek Srikumar. 2024. An in-depth investigation of user response simulation for conversational search. arXiv:2304.07944. Retrieved from <https://arxiv.org/abs/2304.07944>
- [42] Paul Owoicho, Ivan Sekulic, Mohammad Aliannejadi, Jeffrey Dalton, and Fabio Crestani. 2023. Exploiting simulated user feedback for conversational search: Ranking, rewriting, and beyond. In *46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 632–642.
- [43] Ivan Sekulić, Mohammad Alinannejadi, and Fabio Crestani. 2024. Analysing utterances in llm-based user simulation for conversational search. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–22.
- [44] Clemencia Siro, Yifei Yuan, Mohammad Aliannejadi, and Maarten de Rijke. 2024. AGENT-CQ: Automatic generation and evaluation of clarifying questions for conversational search with LLMs. arXiv:2410.19692. Retrieved from <https://arxiv.org/abs/2410.19692>
- [45] Atheer Algherairi and Moataz Ahmed. 2025. Prompting large language models for user simulation in Task-oriented dialogue systems. *Computer Speech & Language* 89 (2025), 101697.
- [46] Sam Davidson, Salvatore Romeo, Raphael Shu, James Gung, Arshit Gupta, Saab Mansour, and Yi Zhang. 2023. User simulation with large language models for evaluating task-oriented dialogue. arXiv:2309.13233. Retrieved from <https://arxiv.org/abs/2309.13233>
- [47] Xiang Luo, Zhiwen Tang, Jin Wang, and Xuejie Zhang. 2024. DuetSim: Building user simulator with dual large language models for task-oriented dialogues. arXiv:2405.13028. Retrieved from <https://arxiv.org/abs/2405.13028>
- [48] Ivan Sekulić, Silvia Terragni, Victor Guimarães, Nghia Khau, Bruna Guedes, Modestas Filipavicius, André Ferreira Manso, and Roland Mathis. 2024. Reliable LLM-based user simulator for task-oriented dialogue systems. arXiv:2402.13374. Retrieved from <https://arxiv.org/abs/2402.13374>
- [49] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2023. A survey on large language model based autonomous agents. arXiv:2308.11432. Retrieved from <https://arxiv.org/abs/2308.11432>
- [50] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative agents for software development. arXiv:2307.07924. Retrieved from <https://arxiv.org/abs/2307.07924>
- [51] Sirui Hong, Xiaowu Zheng, Jonathan Chen, Yuheng Cheng, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, et al. 2023. MetaGPT: Meta programming for multi-agent collaborative framework. arXiv:2308.00352. Retrieved from <https://arxiv.org/abs/2308.00352>
- [52] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration code generation via ChatGPT. arXiv:2304.07590. Retrieved from <https://arxiv.org/abs/2304.07590>
- [53] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI tasks with ChatGPT and its friends in huggingface. arXiv:2303.17580. Retrieved from <https://arxiv.org/abs/2303.17580>
- [54] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. arXiv:2307.16789. Retrieved from <https://arxiv.org/abs/2307.16789>
- [55] Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A benchmark for tool-augmented LLMs. arXiv:2304.08244. Retrieved from <https://arxiv.org/abs/2304.08244>
- [56] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do as I can, not as I say: Grounding language in robotic affordances. arXiv:2204.01691. Retrieved from <https://arxiv.org/abs/2204.01691>
- [57] Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, et al. 2023. Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. arXiv:2305.17144. Retrieved from <https://arxiv.org/abs/2305.17144>
- [58] Kolby Nottingham, Prithviraj Ammanabrolu, Alane Suhr, Yejin Choi, Hannaneh Hajishirzi, Sameer Singh, and Roy Fox. 2023. Do embodied agents dream of pixelated sheep?: Embodied decision making using language guided world modelling. arXiv:2301.12050. Retrieved from <https://arxiv.org/abs/2301.12050>

- [59] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. arXiv:2304.03442. Retrieved from <https://arxiv.org/abs/2304.03442>
- [60] Chen Gao, Xiaochong Lan, Zhihong Lu, Jinzhu Mao, Jinghua Piao, Huandong Wang, Depeng Jin, and Yong Li. 2023. S3: Social-network simulation system with large language model-empowered agents. arXiv:2307.14984. Retrieved from <https://arxiv.org/abs/2307.14984>
- [61] Yancheng Wang, Ziyang Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojiang Huang, Yanbin Lu, and Yingzhen Yang. 2023. RecMind: Large language model powered agent for recommendation. arXiv:2308.14296. Retrieved from <https://arxiv.org/abs/2308.14296>
- [62] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2023. Recommender AI agent: Integrating large language models for interactive recommendations. arXiv:2308.16505. Retrieved from <https://arxiv.org/abs/2308.16505>
- [63] Richard C. Atkinson and Richard M. Shiffrin. 1968. Human memory: A proposed system and its control processes. In *Psychology of learning and motivation*. Vol. 2. Elsevier, 89–195.
- [64] John T. Wixted and Ebbe B. Ebbesen. 1997. Genuine power curves in forgetting: A quantitative analysis of individual subject forgetting functions. *Memory & cognition* 25 (1997), 731–739.
- [65] Richard B. Anderson and Ryan D. Tweney. 1997. Artifactual power curves in forgetting. *Memory & Cognition* 25 (1997), 724–730.
- [66] Andriy Mnih and Russ R. Salakhutdinov. 2007. Probabilistic matrix factorization. *Advances in Neural Information Processing Systems* 20 (2007), 1257–1264.
- [67] Nils Reimers and Iryna Gurevych. 2019. Sentence-Bert: Sentence embeddings using Siamese Bert-networks. arXiv:1908.10084 Retrieved from <https://arxiv.org/abs/1908.10084>
- [68] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805. Retrieved from <https://arxiv.org/abs/1810.04805>
- [69] Shuo Zhang and Krisztian Balog. 2020. Evaluating conversational recommender systems via user simulation. In *26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1512–1520.
- [70] Jiwei Li, Will Monroe, Tianlin Shi, Sébastien Jean, Alan Ritter, and Dan Jurafsky. 2017. Adversarial learning for neural dialogue generation. arXiv:1701.06547. Retrieved from <https://arxiv.org/abs/1701.06547>
- [71] Tien T. Nguyen, Pik-Mai Hui, F. Maxwell Harper, Loren Terveen, and Joseph A. Konstan. 2014. Exploring the filter bubble: The effect of using recommender systems on content diversity. In *23rd International Conference on World Wide Web*, 677–686.
- [72] Jinghua Piao, Jiazhen Liu, Fang Zhang, Jun Su, and Yong Li. 2023. Human–AI adaptive dynamics drives the emergence of information cocoons. *Nature Machine Intelligence* 5, 11 (2023), 1214–1224.
- [73] Nian Li, Chen Gao, Jinghua Piao, Xin Huang, Aizhen Yue, Liang Zhou, Qingmin Liao, and Yong Li. 2022. An exploratory study of information cocoon on short-form video platform. In *31st ACM International Conference on Information & Knowledge Management*, 4178–4182.
- [74] Lei Hou, Xue Pan, Kecheng Liu, Zimo Yang, Jianguo Liu, and Tao Zhou. 2023. Information cocoons in online navigation. *Iscience* 26, 1 (2023), 105893. DOI: <https://doi.org/10.1016/j.isci.2022.105893>
- [75] Xianyangguang Wu. 2022. The influence of social network and information cocoon on major selection. In *2021 International Conference on Social Development and Media Communication (SDMC '21)*. Atlantis Press, 800–804. DOI: <https://doi.org/10.2991/assehr.k.220105.146>
- [76] Zhiyu Kong, Xiaoru Zhang, and Ruilin Wang. 2021. Review of the research on the relationship between algorithmic news recommendation and information cocoons. In *2021 3rd International Conference on Literature, Art and Human Development (ICLAHD '21)*. Atlantis Press, 341–345.
- [77] Hao Ma, Irwin King, and Michael R. Lyu. 2009. Learning to recommend with social trust ensemble. In *32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09)*. ACM, New York, NY, 203–210. DOI: <https://doi.org/10.1145/1571941.1571978>
- [78] Allison J. B. Chaney, David M. Blei, and Tina Eliassi-Rad. 2015. A probabilistic model for using social networks in personalized item recommendation. In *9th ACM Conference on Recommender Systems (RecSys '15)*. ACM, New York, NY, 43–50. DOI: <https://doi.org/10.1145/2792838.2800193>
- [79] Jiliang Tang, Huiji Gao, and Huan Liu. 2012. MTrust: Discerning multi-faceted trust in a connected world. In *5th ACM International Conference on Web Search and Data Mining (WSDM '12)*. ACM, New York, NY, 93–102. DOI: <https://doi.org/10.1145/2124295.2124309>
- [80] Jie Tang, Sen Wu, and Jimeng Sun. 2013. Confluence: Conformity influence in large social networks. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (Chicago, Illinois, USA) (KDD '13)*. Association for Computing Machinery, New York, NY, USA, 347–355. DOI: <https://doi.org/10.1145/2487575.2487691>
- [81] Hongxu Chen, Yicong Li, Xiangguo Sun, Guandong Xu, and Hongzhi Yin. 2021. Temporal meta-path guided explainable recommendation. In *14th ACM International Conference on Web Search and Data Mining*, 1056–1064.

- [82] Juntao Tan, Shuyuan Xu, Yingqiang Ge, Yunqi Li, Xu Chen, and Yongfeng Zhang. 2021. Counterfactual explainable recommendation. In *30th ACM International Conference on Information & Knowledge Management*, 1784–1793.
- [83] Yongfeng Zhang and Xu Chen. 2020. Explainable recommendation: A survey and new perspectives. *Foundations and Trends in Information Retrieval* 14, 1 (2020), 1–101.
- [84] M. Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *Computing Surveys* 55, 7 (2022), 1–38.
- [85] Xinshi Chen, Shuang Li, Hui Li, Shaohua Jiang, Yuan Qi, and Le Song. 2019. Generative adversarial user model for reinforcement learning based recommendation system. In *International Conference on Machine Learning*. PMLR, 1052–1061.
- [86] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, et al. 2023. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104. Retrieved from <https://doi.org/10.48550/arXiv.2312.07104>

Appendices

A Examples of RecAgent

A.1 An Example of the First Step in Sensory Memory

Suppose the observation is a dialogue between David Miller and David Smith as follows:

OB: [David Smith]: Hey David! I recently watched some mind-blowing movies on the recommender system. Have you seen any of these: <Interstellar>, <Inception>, <The Matrix>, <Blade Runner>, or <The Prestige>?

[David Miller]: Oh, hey David! Yes, I actually watched <Interstellar> and <Inception> recently, and they were absolutely amazing! The visuals, storytelling, and mind-bending concepts were mind-blowing. I'm so excited to discuss them with you!

[David Smith]: That's great to hear! I completely agree, those movies are truly mind-blowing. I'm glad you enjoyed them too. I was blown away by the visuals and the way they explore complex ideas. Do you have any other mind-blowing movie recommendations?

[David Miller]: Definitely! If you loved those movies, I think you'll also enjoy <The Matrix>, <Blade Runner>, and <The Prestige>. They have similar mind-bending concepts and incredible storytelling. I can't wait to watch them and discuss them with you!

[David Smith]: Awesome! I've heard great things about those movies as well. Let's make a plan to watch them together and have a movie night. It'll be so much fun to dive into these mind-blowing films and share our thoughts.

[David Miller]: Absolutely! I'm really looking forward to it. We can also invite some other movie enthusiasts to join us and make it a movie marathon. The more, the merrier!

[David Miller]: You're welcome, David. I'm glad too. See you soon and get ready for an unforgettable movie marathon!

Then, we use the following prompt to compress this observation based on LLMs:

Prompt: The observations are as following: {OB}. You should summarize the above observation(s) into one independent sentence. If there is a person's name in the observation, use third person, otherwise use first person. Note that the sentence should pay more attention to the movie interest and the reasons in the observations. The summarization should not include the profile explicitly.

As last, the compressed observation is:

Output: Two individuals, David Smith and David Miller, engage in a conversation about their shared passion for mind-blowing movies, discussing and recommending films such as Interstellar, Inception, The Matrix, Blade Runner, and The Prestige, ultimately planning a movie night and inviting others to join them for a movie marathon.

A.2 An Example of the Insight Generation Process in Short-term Memory

Suppose a memory record

MR: David Miller is interested in mind-bending movies like <Interstellar> and <Inception> and is looking for recommendations from fellow movie lovers to explore more in this genre.

has been enhanced by the following records:

OB: 1. David Smith is recommending the mind-blowing films <Interstellar> and <Inception> that he recently watched on a recommender system, and is seeking further recommendations to explore and discuss.
 2. David Miller enjoyed watching the movies <Interstellar> and <Inception> on the recommender system and found them mind-blowing, prompting him to seek further movie recommendations.
 3. David Smith expressed his interest in movies, particularly mentioning <Interstellar> and <Inception>, seeking recommendations and thoughts from others.

Then, we use the following prompt to generate insights based on LLMs:

Prompt: There are some memories {MR, OB}. Can you infer from the above memories the high-level insight for this person's character? The insight needs to be significantly different from the content and structure of the original memories. Respond in one sentence. Response in one line.

At last, the output is:

Insight: David Miller is a curious and open-minded individual who actively seeks recommendations and discussions about mind-bending movies.

A.3 Example Curves of $g(M_i)$

For an intuitive understanding of the forgotten mechanism in long-term memory, we present several examples of the curve of $g(M_i)$ in Figure A1.

B Prompt Examples for Different Agent Behaviors

The prompt used in RecAgent consists of three parts: a summary of the Agent's personal profile, reaction to given observation, and action-specific instructions.

B.1 Summary

Summary is the process of extracting and summarizing information relevant to the current observation from the user's profile.

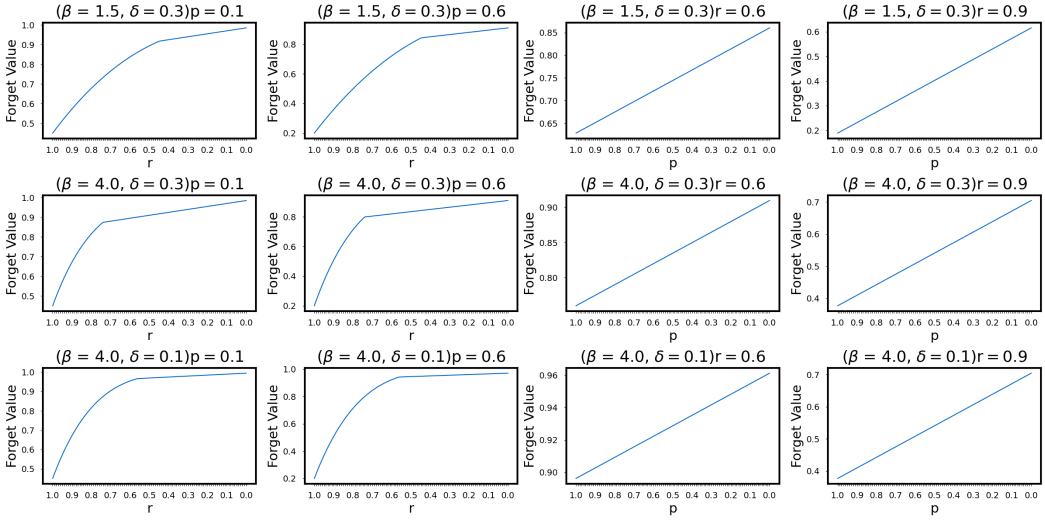


Fig. A1. Examples of the curve of $g(M_i)$ with different hyper-parameters.

Summary: Given the following observation about David Miller: “David Smith is going to chat with David Miller.” Please summarize the relevant details from his profile. His profile information is as follows:

Name: David Miller

Age: 39

Gender: female

Traits: fun-loving, creative, practical, energetic, patient

Status: writer

Movie Interest: action movies, sci-fi movies, classic movies

Feature: Choose movies, enjoy watching, and provide feedback and ratings to the recommendation system. Search for movies heard of before and expand movie experiences...Enjoy publicly posting on social media and sharing content and insights with more people.

Interpersonal Relationships: ‘David Smith’: ‘friend’

Please avoid repeating the observation in the summary.

Summary:

Output: David Smith is a 25-year-old male photographer who is compassionate, caring, ambitious, and optimistic. He has an interest in sci-fi and comedy movies and enjoys choosing, watching, and providing feedback and ratings to the recommendation system. He has high standards for movies and the recommendation system and may criticize both. He has a friendship with David Miller.

B.2 Reaction

This section serves as a shared prompt framework for all actions, integrating information such as summary, memory, observation, and more to obtain the agent’s reaction. It covers reactions for both individual agent actions and dialogues involving two agents.

– Single Agent Reaction

Reaction: Name: David Smith (age: 25) {Summary}

It is September 12, 2023, 08:00 AM.

David Smith recently heard [‘The Matrix’, ‘Back to the Future’, ‘Ánchorman’, ‘Superbad’] on social media.

David Miller recently watched nothing on recommender system. Other than that David Miller doesn’t know any movies.

Most recent observations: David and Millar had a conversation about their shared interest in movies, discussing their favorite genres such as sci-fi, comedy, and action, exchanging recommendations, expressing their skepticism about recommendation systems, and making plans to compare notes and potentially organize a movie night together. David Miller is seeking recommendations for movies such as “The Matrix,” “Back to the Future,” “Anchor-man,” and “Superbad” on social media, expressing interest in planning a movie night and inviting others to share their thoughts and suggestions.

Observation: David Miller want to post for all acquaintances.

All occurrences of movie names should be enclosed with <>

– Reaction Between Two Agent

Reaction: Name: David Smith (age: 25)

Summary

Name: David Miller (age: 39)

Summary

It is September 12, 2023, 10:28 PM.

David Smith recently heard [‘The Matrix’, ‘Back to the Future’, ‘Ánchorman’, ‘Superbad’] on social media.

David Smith recently watched nothing on recommender system.

Other than that David Smith doesn’t know any movies.

David Miller recently heard [‘Inception’, ‘The Dark Knight’] on social media. David Miller recently watched nothing on recommender system. Other than that David Miller doesn’t know any movies.

Most recent observations of David Smith: David Smith observed that David Miller was interested in chatting about movies, as evidenced by his action of initiating a chat conversation. David and David had a conversation about their mutual interest in movies, with David Miller recommending sci-fi and action films like “Inception” and “The Dark Knight” to David Smith, who expressed interest in watching them and sharing his thoughts with David Miller in the future.

Most recent observations of David Miller: David Miller and David Smith had a conversation about their shared interest in movies, with David Miller recommending sci-fi movie “Inception” and action movie “The Dark Knight” to David Smith, who expressed interest in watching them and discussing them later.; David Miller expressed his excitement about two highly recommended movies, “Inception” and “The Dark Knight,” which he discovered on social media, and eagerly anticipates watching them and sharing his reviews with others.;

Observation: David Smith is going to chat with David Miller.

All occurrences of movie names should be enclosed with <>.

B.3 Action

This section includes all the actions that agents can perform, along with examples of input and output for each action.

– Take Action

OB: David Miller must take only ONE of the actions below: (1) Enter the Recommender System. If so, David Miller will be recommended some movies, from which David Miller can watch some movies, or search for movies by himself.

(2) Enter the Social Media. David Miller can chat with friends or publish a post to all friends of David Miller. If David Miller recently watched some movies they might want to share with others.

(3) Do Nothing.

Suffix: What action would David Miller like to take? Respond in one line.

If David Miller wants to enter the Recommender System, write:

[RECOMMENDER]:: David Miller enters the Recommender System

If David Miller wants to enter the Social Media, write:

[SOCIAL]:: David Miller enters the Social Media

If David Miller wants to do nothing, write:

[NOTHING]:: David Miller does nothing

Output: [RECOMMENDER]:: David Miller enters the Recommender System.

– Take Recommender Action

OB: David Smith is browsing the recommender system. David Smith is recommended [`<Eye of Vichy, The (Oeil de Vichy, L')>`||`The movie <Eye of Vichy, The (Oeil de Vichy, L')>` is a documentary that delves into the propaganda films made by the Vichy government during World War II.”, `<Oliver!>`||`<Oliver!>` is a musical adaptation of Charles Dickens’ classic novel `<Oliver Twist>`.”, `<...And Justice for All>`||`<...And Justice for All>` is a legal drama film from 1979 that follows the story of an ethical and idealistic defense attorney named Arthur Kirkland, played by Al Pacino. Kirkland finds himself in a moral dilemma when he is forced to defend a judge who he despises, who has been accused of rape. As he navigates the corrupt legal system and grapples with his own values, Kirkland must decide whether to compromise his integrity or risk losing everything he has worked for. The film explores themes of justice, morality, and the flaws of the American legal system.”, `<T-Men>`||`<T-Men>` is a film noir about two Treasury agents who go undercover to break up a counterfeiting ring.”, `<Son of Flubber>`||`<Son of Flubber>` is a 1963 American comedy film directed by Robert Stevenson and starring Fred MacMurray.”].

Suffix: David Miller must choose one of the four actions below: (1) Watch ONLY ONE movie from the list returned by the recommender system.

(2) See the next page.

(3) Search for a specific item.

(4) Leave the recommender system.

If David Miller has recently heard about a particular movie on social media, David Miller might want to search for that movie on the recommender system. To watch a movie from the recommended list, write: [BUY]:: ONLY ONE movie name||description. To see the next page, write: [NEXT]:: David Miller views the next page. To search for a specific item, write: [SEARCH]:: single, specific movie name to search for. To leave the recommender system, write: [LEAVE]:: David Miller leaves the recommender system.

Output: [BUY]::<Son of Flubber>||<Son of Flubber> is a 1963 American comedy film directed by Robert Stevenson and starring Fred MacMurray.

– Generate Feeling

OB: David Jones has just finished watching S.F.W.;;S.F.W. (1994) is a dark comedy-drama film about a group of teenagers who are held hostage in a convenience store by a media-savvy, charismatic, and nihilistic young man, Cliff Spab (played by Stephen Dorff), and his equally disillusioned girlfriend, Wendy (played by Reese Witherspoon). The film explores themes of media manipulation, youth disillusionment, and the commodification of tragedy in modern society.

Suffix: David Jones, how did you feel about the movie you just watched? Describe your feelings in one line.NOTE: Please answer in the first-person perspective.

Output: I found “SFW” to be a thought-provoking and captivating exploration of media manipulation and youth disillusionment in modern society.

– Search Item

OB: David Miller is browsing the recommender system. David Miller has searched for Interstellar in recommender system and recommender system returns item list:[“Supernova (2000)”, “October Sky (1999)”, “Cat from Outer Space, The (1978)”, “Space Cowboys (2000)”, “Aliens (1986)”] as search results.

Suffix: David Miller must choose one of the four actions below:

- (1) Watch ONLY ONE movie from the list returned by the recommender system.
- (2) See the next page.
- (3) Search for a specific item.
- (4) Leave the recommender system.

If David Miller has recently heard about a particular movie on social media, David Miller might want to search for that movie on the recommender system.

To watch a movie from the recommended list, write:

[BUY]:: ONLY ONE movie name;;description

To see the next page, write:

[NEXT]:: David Miller views the next page.

To search for a specific item, write:

[SEARCH]:: single, specific movie name to search for.

To leave the recommender system, write:

[LEAVE]:: David Miller leaves the recommender system.

Output: [SEARCH]:: Inception

— Generate Dialogue

OB: David Smith is chatting with David Miller.

Suffix: What will be said between David Smith and David Miller? David Smith initiates the conversation first. Please simulate their conversation.

David Smith and David Miller should not say anything about movies they have not watched or heard about.

Write the dialogue in the following format:

[David Smith]:

[David Miller]:

Output: [David Smith]: Hey David! How's it going? I heard you were interested in a movie. What's been on your mind?

[David Miller]: Hey David! I'm doing great, thanks for asking. Yeah, I've been hearing a lot about this movie <Interstellar> recently. Have you heard of it too?

[David Smith]: Absolutely! Actually, I've been seeing it all over social media as well.

[David Miller]: That's awesome! I'm glad you enjoyed it. I've been wanting to watch it too. Would you be up for a cozy movie night to watch it together? We can discuss our thoughts and interpretations afterwards.

[David Smith]: I'd love that! It's always more fun to watch movies with friends and have those deep conversations afterwards. Count me in!

[David Miller]: Great! I'll make sure to set up a movie night soon. By the way, have you heard of any other movies that you'd recommend? I'm always open to expanding my movie experiences.

– Publish Posting

OB: David Smith want to post for all acquaintances.

Suffix: Posts should be related to recent watched movies on recommender systems. David Smith should not say anything about movies that have not watched or heard about. If you were David Smith, what will you post? Respond in one line.

Output: Hey everyone! Just watched <Inception> on the recommender system and it was absolutely mind-blowing! Highly recommend checking it out!

Received 19 March 2024; revised 26 November 2024; accepted 16 December 2024